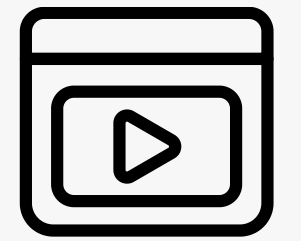


언리얼엔진 포트폴리오



Github Link



Game Play
Youtube Link
& QR Cord

Contact

- Email : uk7732@naver.com
- Nama : 박동욱
- Phone : 01021309654

목차

1) System Architecture & TID Data Flow



0-0 게임 소개 - 4p [Link](#)

1-3 Spawn System - 16p [Link](#)

1-0 Class Diagram - 5p [Link](#)

1-4 Skill System - 21p [Link](#)

1-1 TableSubsystem - 6p [Link](#)

1-5 Ability System - 26p [Link](#)

1-2 CharacterBase - 13p [Link](#)

1-6 Item/Inventory/
Equipment - 32p [Link](#)

목차

2) In-Game Content & Visual Elements



2-1 레벨 구성 - 42p [Link](#)

2-4 캐릭터
(플레이어 & 몬스터) - 52p [Link](#)

2-2 Widget - 44p [Link](#)

2-5 스킬인디케이트 - 56p [Link](#)

2-3 룸 상태 관리 - 47p [Link](#)

2-6 오클루전 컴포넌트 - 59p [Link](#)

게임 소개

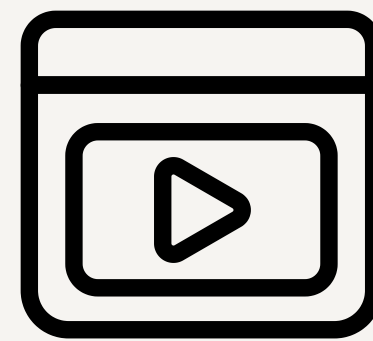
Project Name : Sleek Robot

확장성을 고려한 쿼터뷰 게임 설계

본 프로젝트는 쿼터뷰 시점을 기반으로 한 **PC 멀티 액션 게임**으로 직관적인 조작과 안정적인 플레이 흐름을 목표로 개발되었습니다.

핵심 조작에 집중할 수 있도록 입력 구조를 단순화하고
향후 콘텐츠 확장과 기획 변경에 유연하게 대응 할 수 있는 설계에
중점을 두었습니다.

특히 **데이터 구동 방식(Data-Driven)**을 채택하여
새로운 캐릭터, 스킬, 아이템 등의 콘텐츠 볼륨을 늘릴 때
추가적인 c++코드 수정이나 재컴파일 없이
에셋과 데이터 테이블 편집만으로 신속하게 찍어낼 수 있는
확장성 높은 구조를 구축했습니다.



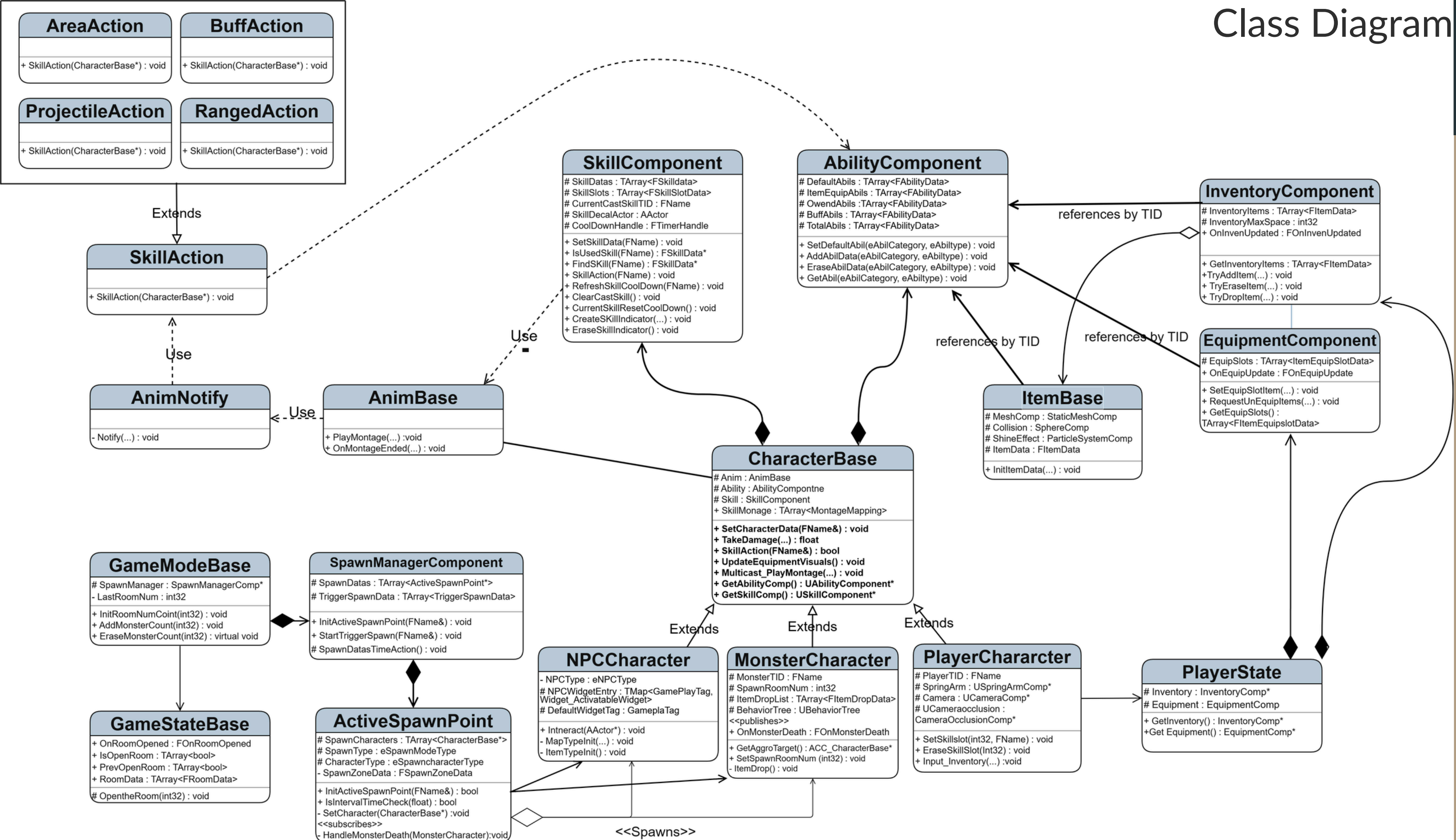
링크

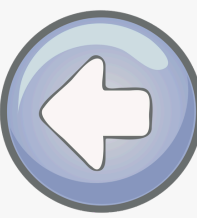


QR

게임 하이라이트 영상

Class Diagram





1-1 TableSubsystem



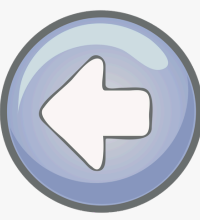
본 단원에서는

본 게임 전반에 사용되는 **데이터 테이블 시스템의 핵심 기반인 Table Subsystem**과 이를 보조하는 **TableDatas** 상수 정의 구조에 대해 설명합니다.

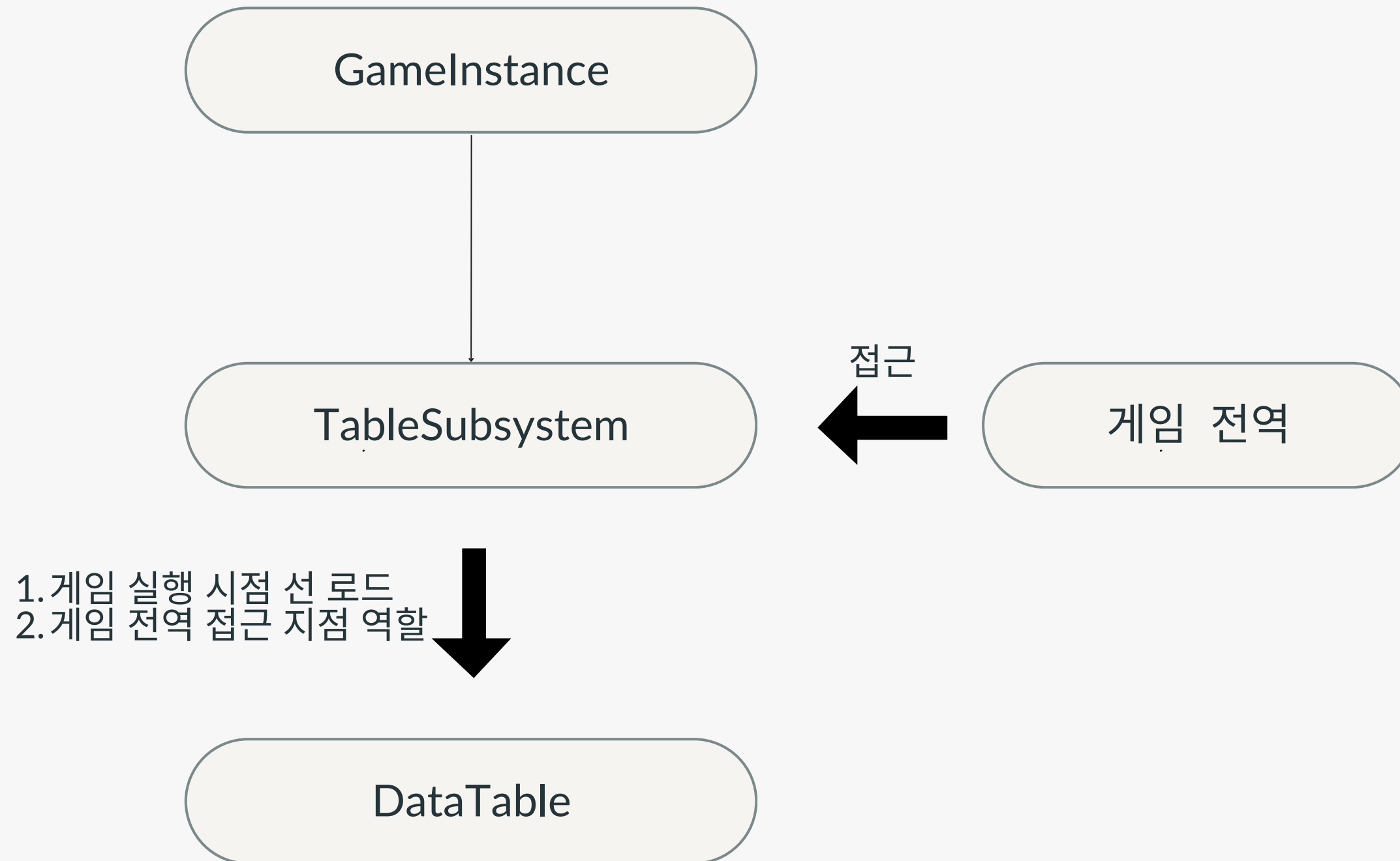
본 시스템은 Unreal Engine의 **DataTable**을 중심으로

게임 전반의 데이터를 코드로부터 분리하고

런타임에서 안정적이고 일관되게 접근할 수 있도록 설계되었습니다



TableSubsystem의 역할과 책임



GameInstance 기반 전역 데이터 관리자

TableSubsystem은 GameInstanceSubsystem을 상속받아 구현되었으며
게임 실행 시점에 필요한 모든 **데이터를 선 로드**하고
이후 게임 전역에서 **접근 지점 역할**을 수행합니다.

이를 통해

- 테이블 로딩 중복 방지
- 월드 / 씬 전환과 무관한 데이터 유지
- 데이터 접근 경로 통일

을 동시에 달성합니다.



초기 테이블 로딩 흐름

 TableLoad			
	행 이름	Table Path	Is Load
1	AbilCategoryTable	/Game/TableData/AbilCategoryTable	True
2	AbilGroupTable	/Game/TableData/AbilGroupTable	True
3	AbilTable	/Game/TableData/AbilTable	True
4	AITable	/Game/TableData/AITable	True
5	CoinTable	/Game/TableData/CoinTable	True
6	DefineTable	/Game/TableData/DefineTable	True
7	ItemEnchantTable	/Game/TableData/ItemEnchantTable	True
8	ItemEquipTable	/Game/TableData/ItemEquipTable	True
9	ItemTable	/Game/TableData/ItemTable	True
10	MapTable	/Game/TableData/MapTable	True
11	MonsterTable	/Game/TableData/MonsterTable	True
12	PlayerTable	/Game/TableData/PlayerTable	True
13	RewardTable	/Game/TableData/RewardTable	True
14	SkillTable	/Game/TableData/SkillTable	True
15	SpawnTable	/Game/TableData/SpawnTable	True
16	TagTable	/Game/TableData/TagTable	True
17	TeamTable	/Game/TableData/TeamTable	True
18	AnimationTable	/Game/TableData/AnimationTable	True
19	SpawnGroupTable	/Game/TableData/SpawnGroupTable	True
20	NPCTable	/Game/TableData/NPCTable	True
21	MapGroupTable	/Game/TableData/MapGroupTable	True

TableSubsystem은 다음과 같은 순서로 동작합니다.

1. 게임 시작시 Initialize()호출



2. TableLoad전용 테이블 로드



3. TableLoad에 명시된 경로 기반으로 실제 테이블 로드



4. 로드된 테이블을 TMap<FName, UDataTable> 상태로 캐싱



5. 이후 런타임에서는 캐싱된 테이블만 참조

어떤 테이블을 로드할 것인지는 데이터로 관리하고 실제 접근은 코드에서 통제하는 구조입니다.



TableDatas 구조 분리 설계

[TableSubsystem]

역할 : 로딩과 접근 책임

- 테이블 로드 시점 관리
- 테이블 캐싱 및 생명주기 관리
- 런타임 데이터 접근 API 제공

어떻게 테이블을 불러오고
관리하는가에 집중

[TableDatas]

역할 : 데이터 식별자와 이름 규약

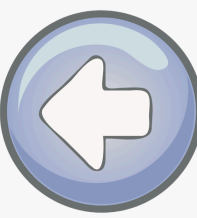
TableDatas는
게임에서 사용되는 모든 테이블 이름과
공통Define 키 값을 FName상수로
중앙 관리 합니다.

본 프로젝트의 데이터 테이블 관련 정의는
TableSubsystem과 TableDatas 두 영역으로 분리되어
있습니다.

이 분리를 통해

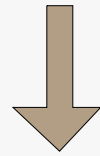
- 데이터 구조 변경 시 로직 영향 최소화
- 테이블 추가/삭제에 대한 확장성 확보
- 코드 가독성과 유지보수성 향상

이라는 설계 효과를 얻을 수 있었습니다.

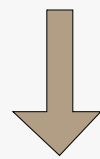


데이터 테이블 접근방식

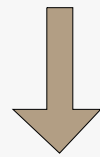
Game Logic(Skill/Item/Monster 등등 클래스)



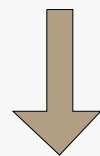
FindTableRow(TableName, TID) *TID = Table ID



TableSubsystem



Cached DataTable (TMap)



Row Pointer / Value 반환



```
UTableSubsystem* UTableSubsystem::Get(const UObject* worldContext)
{
    if (nullptr == worldContext) return nullptr;

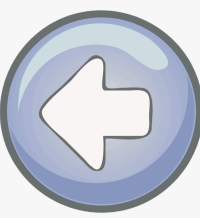
    UGameInstance* inst = UGameplayStatics::GetGameInstance(worldContext);
    if (nullptr == inst) return nullptr;

    return inst->GetSubsystem<UTableSubsystem>();
}
```

***UTableSubsystem::Get()** 을 통해 모든 게임 로직에서
TableSubsystem에 접근할 수 있으며
실제 데이터 조회는 **FindTableRow()**를 통해 수행함

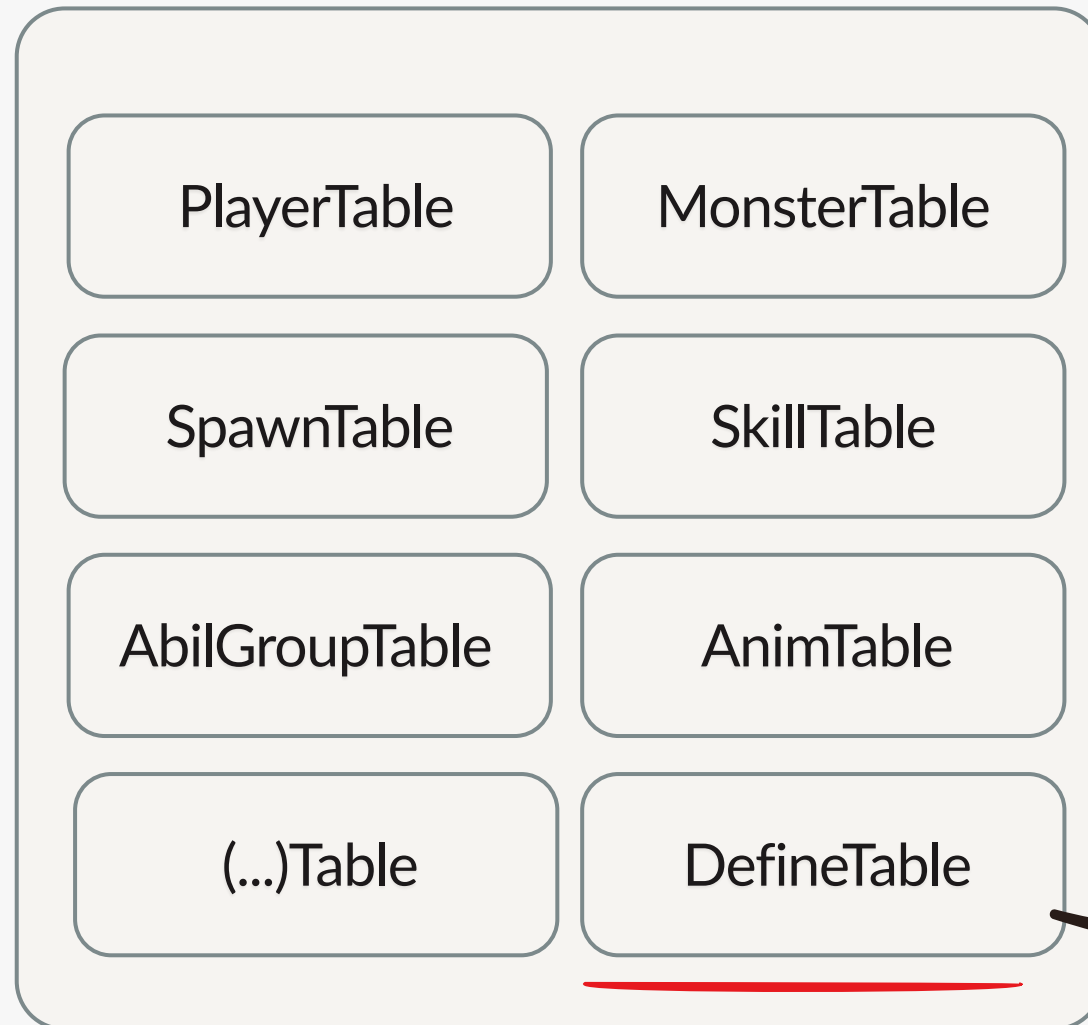
- Game Logic은 DataTable을 직접 모름
- 모든 화살표는 Subsystem을 거침
- DataTable은 외부에 노출되지 않음

모든 게임 로직이 Table Subsystem Find()를 통해 데이터에 접근하도록 설계하였습니다.



Define Table 활용

1. 테이블 단위

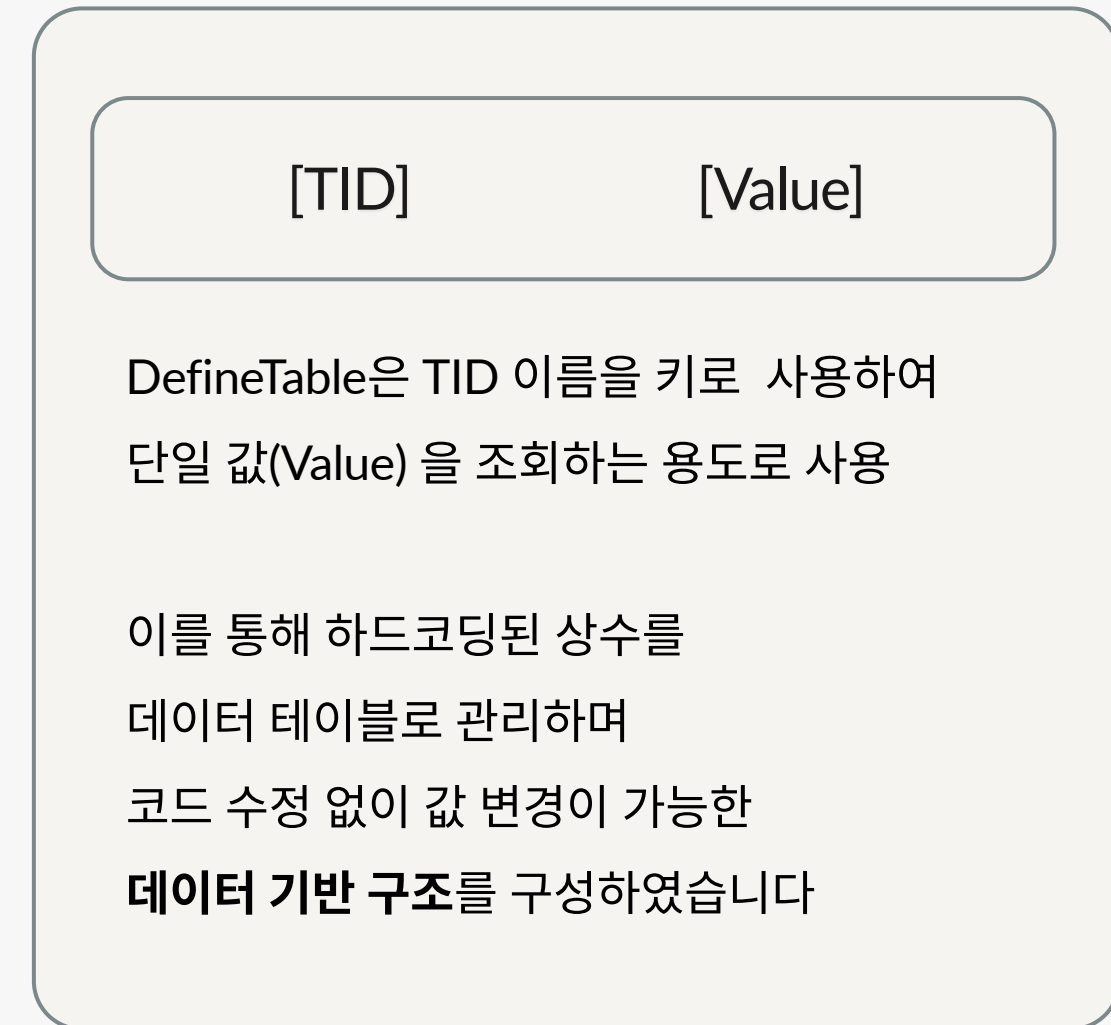


몬스터, 스킬, 아이템 등과 같이
엔티티 단위의 정보 묶음을 관리할때 사용합니다.

특징

- 하나의 테이블이 하나의 데이터 도메인을 담당
- Row ID 기반으로 데이터 조회
- 런타임 로직에서 가장 일반적으로 사용

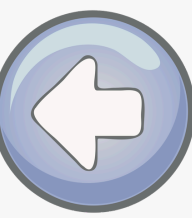
2. Define(상수)기반



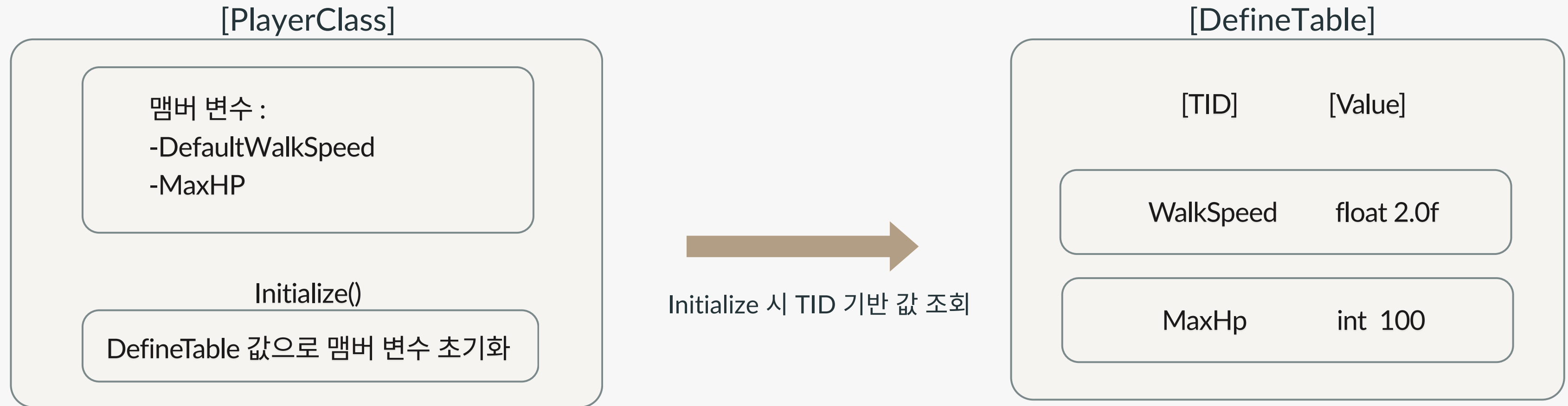
Row ID를 변수처럼 정의하여
값 자체를 테이블로 관리하는 구조

특징

- Row ID가 곧 의미있는 상수 역할
- 코드 전반에서 공통으로 사용
- 밸런싱 및 기획 수정등에 사용



Define Table 활용 예시



- 수도 코드

```
Initialize()  
{  
    DefaultWalkSpeed = DefineTable[WalkSpeed]  
    MaxHP = DefineTable[MaxHP]  
}
```

코드 내 상수를 수정하지 않고도 테이블 값 변경만으로
변수 제어가 가능하도록 구성하였습니다.



1-2 characterBase

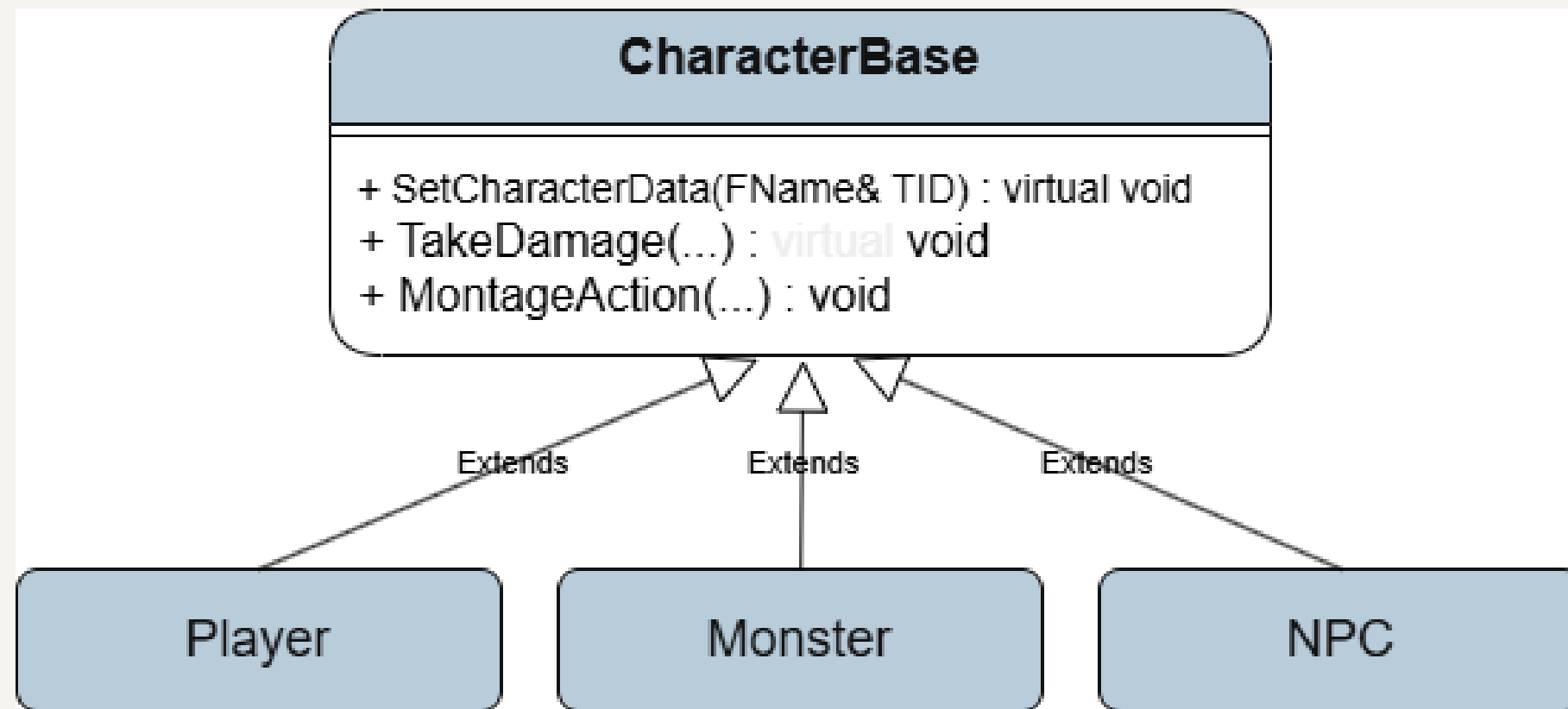


CharacterBase는

Player, Monster, NPC가 공통으로 사용하는 최상위 캐릭터
베이스 클래스입니다.

이 클래스는 Ability, Skill등의 핵심 시스템 컴포넌트를 조합
하는 역할만 담당합니다

시스템 로직은 재사용 되고 캐릭터 클래스는 행동 정의에만
집중할 수 있도록 설계했습니다.



- 부모 클래스에서 캐릭터의 공통 로직을 구현하여 코드 중복을 최소화 했습니다
- 각 캐릭터(자식클래스)마다 다르게 동작해야 하는 핵심기능 (SetCharacterData 등) 은 가상함수로 선언하여 확장성을 확보 했습니다

Player

NewRow

Player Class	None	↶ 🔍 + ✕
Tag TID	None	
Abil Group TID	None	
Skill IDs	배열 엘리먼트: 0	+ 🗑
Team ID	0	
Animation TID	None	

Monster

NewRow

Monster Class	None	↶ 🔍 + ✕
Monster Name	None	
Tag TID	None	
Abil Group TID	None	
Skill IDs	배열 엘리먼트: 0	+ 🗑
Animation TID	None	

NPC

Type	NPC_Map	↕
NPCClass	None	↶ 🔍 + ✕
Name	None	
Dialogue ID		🚩
▼ NPCTIDDatas		
Map Group TID	None	
ITem Shop TID	None	
Tag TID	None	
Animation TID	None	
Widget Entry	배열 엘리먼트: 0	+ 🗑
Start Widget Tag	None	↕

캐릭터 데이터는 데이터테이블로 정의되며 각 캐릭터는 TID를 기준으로 참조 및 생성됩니다.





1-3 Spawn System



스폰 시스템은 코드 내 하드코딩을 배제하고 데이터 테이블의 TID를 기반으로 동작하며
관리 효율과 확장성을 높이기 위해 2단계 계층 구조로 구현되었습니다.

- USpawnManagerComponent (총괄 관리): 게임 시작 시 TID 테이블을 읽어와 다수의 스폰 영역(UActiveSpawnPoint)을 동적으로 생성하고 관리합니다.
- UActiveSpawnPoint (개별 스폰): 매니저로부터 생성되어 각 영역의 규칙(랜덤 범위, 스폰 주기 등)에 맞춰 실제 캐릭터(몬스터/NPC)를 스폰하고 생명주기를 제어합니다.

모든 스폰 요청은 서버 RPC 검증을 거쳐 처리되며
네비게이션 메시지를 활용해
캐릭터가 지형에 끼이지 않는 유효한 위치에 배치되도록 보장합니다.



[1. SpawnTable]

▼ SpawnTable		
Type	None ▾	
Character Type	Spawn_None ▾	
Spawn Name	None	
Character TID	None	
▶ Spawn Location	0.0	0.0
▶ Spawn Rotation	0.0 °	0.0 °
Spawn Radius or Side	0.0	
Spawn Count	0	
Spawn Room Number	0	
Spawn Interval	0.0	
Is Trigger Spawn	☐	

스폰시스템 데이터 테이블 설계 목적

- 스폰 데이터의 관리 효율성을 높이기 위해 스폰 그룹 테이블과 스폰 테이블을 분리하여 설계했습니다.

1. 스폰 테이블 : 개별 몬스터의 스폰 조건 및 데이터를 관리하는 기본단위입니다. 특정 스폰 정보를 다른 구역이나 모드에서도 손쉽게 재사용 할 수 있도록 독립적으로 구성했습니다.

2. 스폰 그룹 테이블 : 여러 스폰 데이터를 하나의 그룹으로 묶어주는 컨테이너 역할을 합니다. 이를 통해 게임모드나 스테이지 별로 필요한 스폰 리스트를 모듈화하여 손쉽게 교체 및 관리할 수 있습니다.

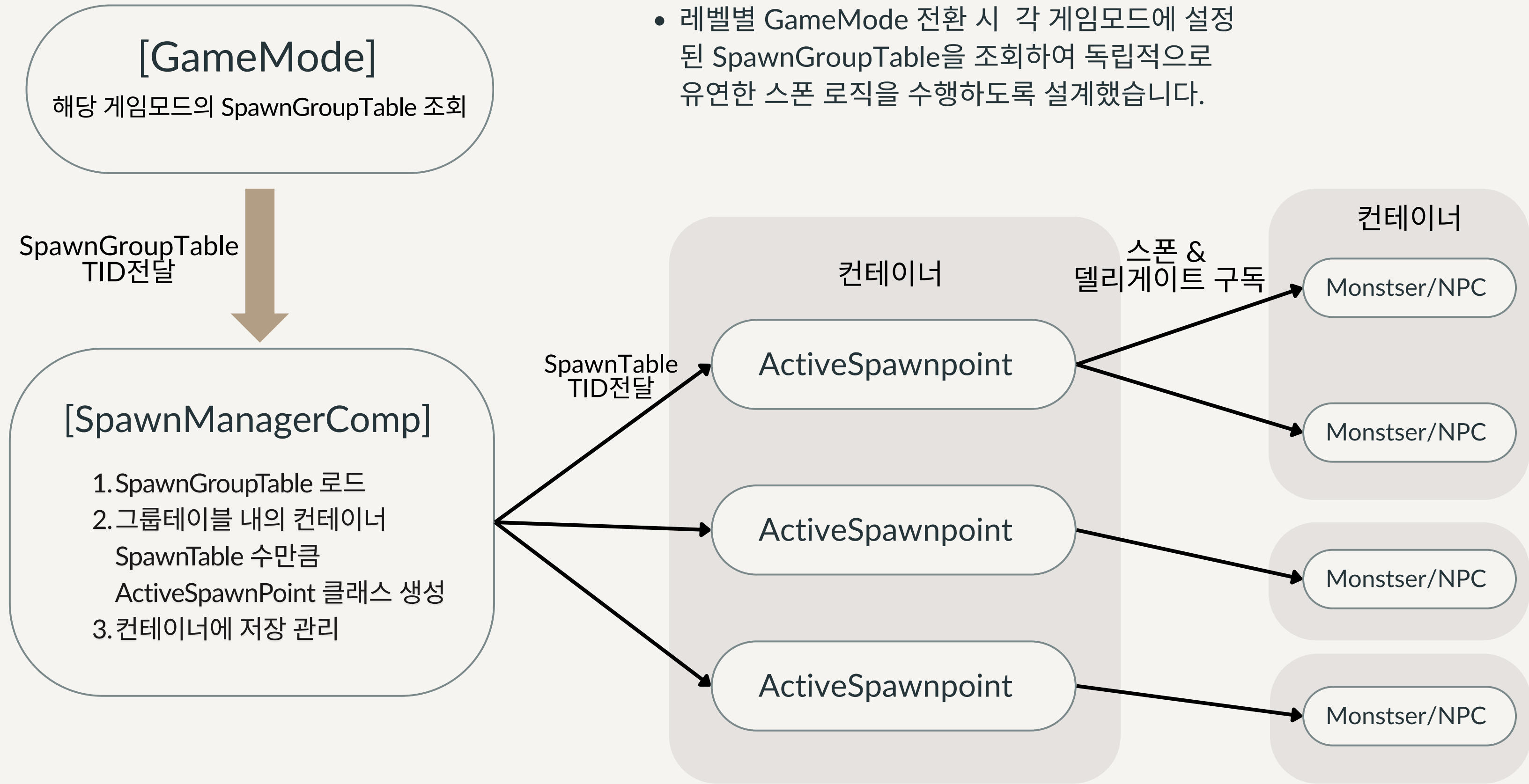
[2. SpawnGroupTable]

▼ SpawnGroupTable		
Spawn Table TIDs	배열 엘리먼트: 0	⊕ ⊞



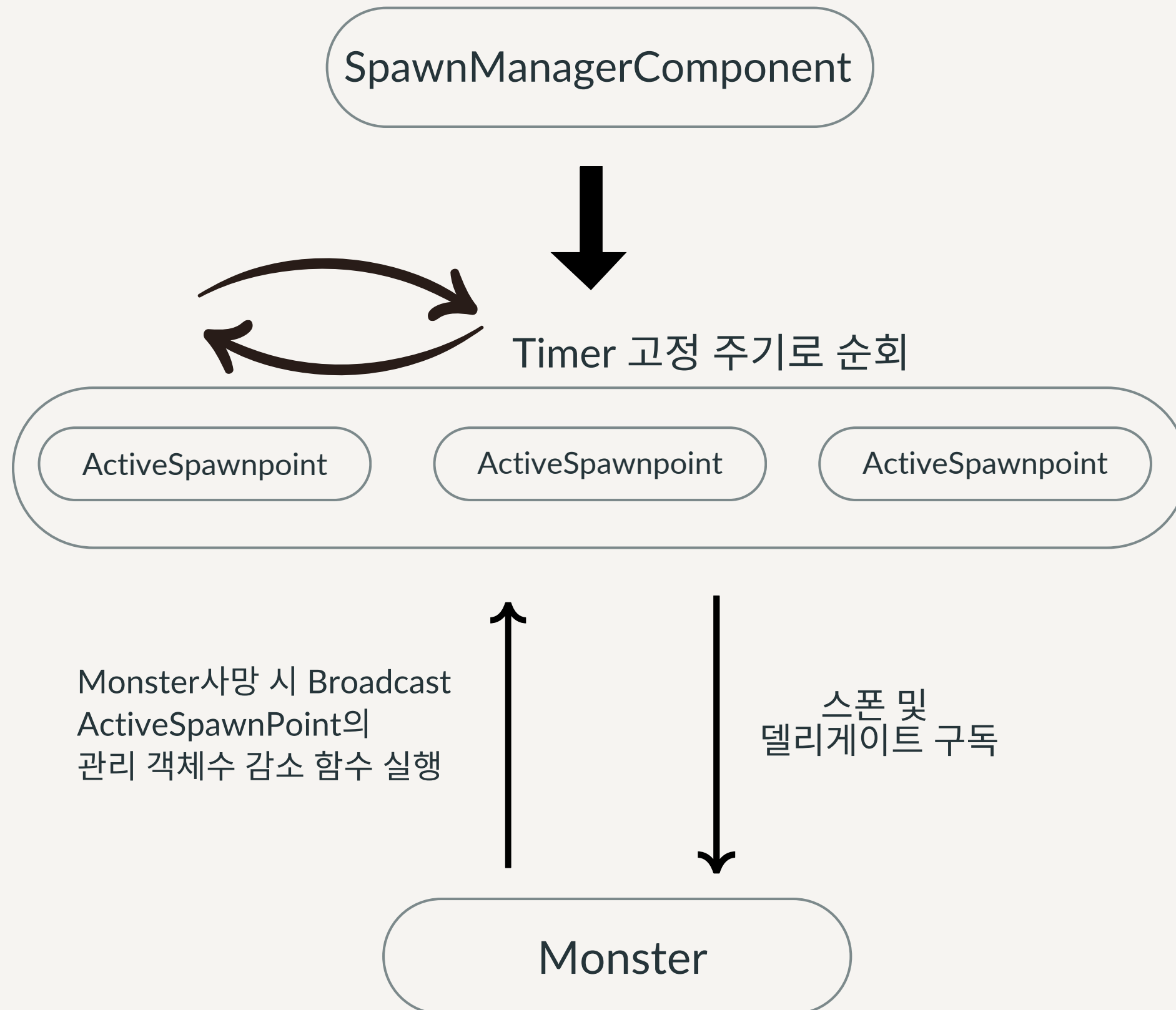
스폰 시스템 구조도

- 레벨별 GameMode 전환 시 각 게임모드에 설정된 SpawnGroupTable을 조회하여 독립적으로 유연한 스폰 로직을 수행하도록 설계했습니다.





Timer 기반 스케줄링 + Event-Driven 반응 구조



스폰 시스템은 Timer 기반의 주기적 스케줄링과 이벤트 기반 (Event Driven) 반응 구조를 결합하여 설계 하였습니다.

- SpawnManagerComponent는 타이머를 통해 ActiveSpawnPoint들을 주기적으로 검사하며 각 ActiveSpawnPoint는 자신의 상태를 기준으로 스폰 가능 여부를 판단합니다. 이를 통해 상시 Tick 사용을 제거하고 스폰 타이밍을 제어합니다.
- 몬스터 생성 이후의 생명 주기 관리는 ActiveSpawnPoint가 담당하며 몬스터 사망 시에는 Delegate 이벤트를 통해 즉시 스폰 수를 갱신합니다.



[SpawnTable]

Spawn Radius or Side	0.0
Spawn Count	0
Spawn Room Number	0
Spawn Interval	0.0
Is Trigger Spawn	☐

인터벌 스폰과 트리거 스폰

1.인터벌 스폰 : 주기적인 상태 체크를 통해 필드의 몬스터 개체수를 일정하게 유지하는 방식입니다.

- 동작 로직 : 지정된 시간(Interval)마다 해당 스폰 구역 내의 몬스터 수량을 체크합니다.
- 조건 충족 시 : 필드에 남아 있는 몬스터가 기획된 최 대 수량이상이면 스폰을 건너뛵니다.
- 조건 미달 시 : 몬스터가 처치되어 최대수량보다 부족해지면 부족한 수량만큼 재스폰을 수행합니다.

2.트리거 스폰 : 특정 이벤트나 플레이어의 상호작용에 의 해서만 몬스터가 생성되는 방식입니다.

- 동작 로직 : IsTriggerSpawn 옵션이 활성화 된 경우 주기적인 자동 스폰 루프에서 제외됩니다. 이후 명시적인 스폰 요청이 들어 오는 시점에만 일회성 또는 조건부로 스폰합니다



1-4 Skill System

SkillComponent는 스킬 사용 흐름을 제어하는 컨트롤러 역할을 담당합니다.

- TID 확인
- 스킬 사용 가능 여부 검증
- 몽타주 실행

공통 처리만 담당하며 스킬의 실제 실행 로직은 SkillAction 클래스로 분리했습니다.



스킬 시스템 데이터 흐름도

01 스킬 발동 요청

Character클래스에서 SkillComponent
멤버변수의 SkillAction()호출
(클라이언트 요청)



02 검증 및 몽타주 재생

1. server_SkillAction()실행
2. 데이터 기반 스킬 조회(TID)
3. 스킬 사용 가능 여부 검증
4. 플레이어 / 몬스터 분기처리
5. 해당 스킬 애니메이션 몽타주 검증 후 실행



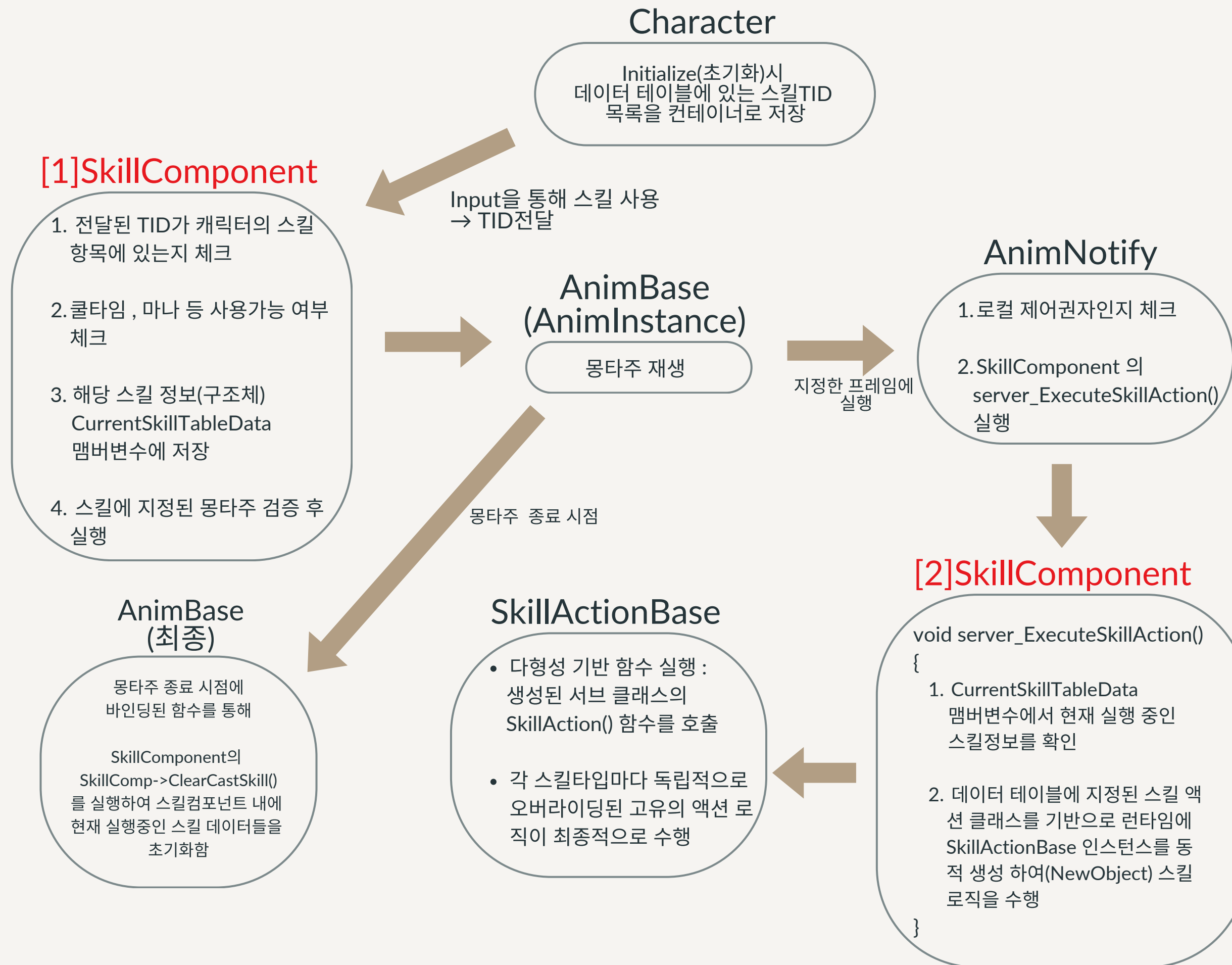
03 AnimNotify 트리거

- 몽타주 재생 중 특정 프레임에서 애니메이션 노티파이 발동
- 스킬컴포넌트의 Server_ExecuteSkillAction() 역호출



04 다형성 기반 스킬로직 동적 실행

- NewObject를 통해 SkillActionBase 인스턴스 동적 생성
- SkillActionBase 자식클래스로 분기 및 스킬 최종 처리



본 구조도는 스킬 데이터(TID), 재생 타이밍(애니메이션 프레임), 실제 로직(SkillAction)의 결합도를 분리한 레이어드 아키텍처형 스킬시스템입니다.

- 설계 이유 : TID 중심의 깔끔한 데이터 테이블 구조를 유지하면서도 애니메이션의 원하는 특정 프레임(타이밍)에 스킬의 실제 로직을 정확히 연동하여 발동시키기 위해 설계했습니다.

아키텍처 기대효과

1. 확장성 : 컴포넌트 코드 수정 없이 SkillActionBase를 상속한 신규 액션 클래스 구현과 데이터 테이블 등록만으로 새로운 스킬을 손쉽게 확장할 수 있습니다.
이는 개방-폐쇄 원칙을 자연스럽게 만족하는 구조입니다.
2. 느슨한 결합도 : 애니메이션과 스킬로직이 오직 TID 이름표로만 소통하므로 서로 영향을 주지 않아 유지보수가 독립적입니다.
3. 명확한 생명주기 관리 : 스킬 캐스팅 시작부터 애니메이션 종료 시점까지의 데이터 흐름과 메모리 해제 타이밍이 안전하게 자동으로 제어됩니다.



```
void UAnimNotify_SkillAction::Notify(...)
{
    Super::Notify(MeshComp, Animation, EventReference);
    ACC_CharacterBase* character = Cast<ACC_CharacterBase>(MeshComp->GetOwner());

    if (character)
    {
        USkillComponent* SkillComp = character->GetSkillComp();
        if (SkillComp)
        {
            SkillComp->EraseSkillIndicator(); // 1. 시각적 인디케이터 제거

            if(character->IsLocallyControlled())
                SkillComp->Server_ExecuteSkillAction(); // 2. 로컬 제어권자 확인 후
                                                         // 서버 호출 (RPC 역호출)
        }
    }
}
```

단일 AnimNotify 기반 스킬 실행 트리거

- 스킬마다 별도의 AnimNotify를 생성하지 않고 AnimNotify_SkillAction 단 하나를 통해 모든 스킬의 실행 시점을 제어하는 구조로 설계되었습니다.
- AnimNotify는 특정 프레임에서 스킬이 발동되어야한다는 시점 정보만 전달하며 실제 스킬 로직이나 스킬 종류에 대해서는 전혀 알지 않습니다.

느슨한 결합과 책임 분리

- 애니메이션 몽타주 : 스킬이 발동되어야 할 타이밍만 정의
- AnimNotify_SkillAction : 애니메이션으로부터 전달받은 시점을 시전자의 SkillComponent로 전달하는 중간 트리거 역할
- SkillComponent : 현재 캐스팅 중인 스킬 상태를 보유하며 실제 실행 로직(SkillAction)을 구동

이를 통해 애니메이션, 실행 타이밍, 스킬 로직 간의 결합도를 최소화한 구조를 유지합니다.

조건부 RPC 기반 서버 실행 요청

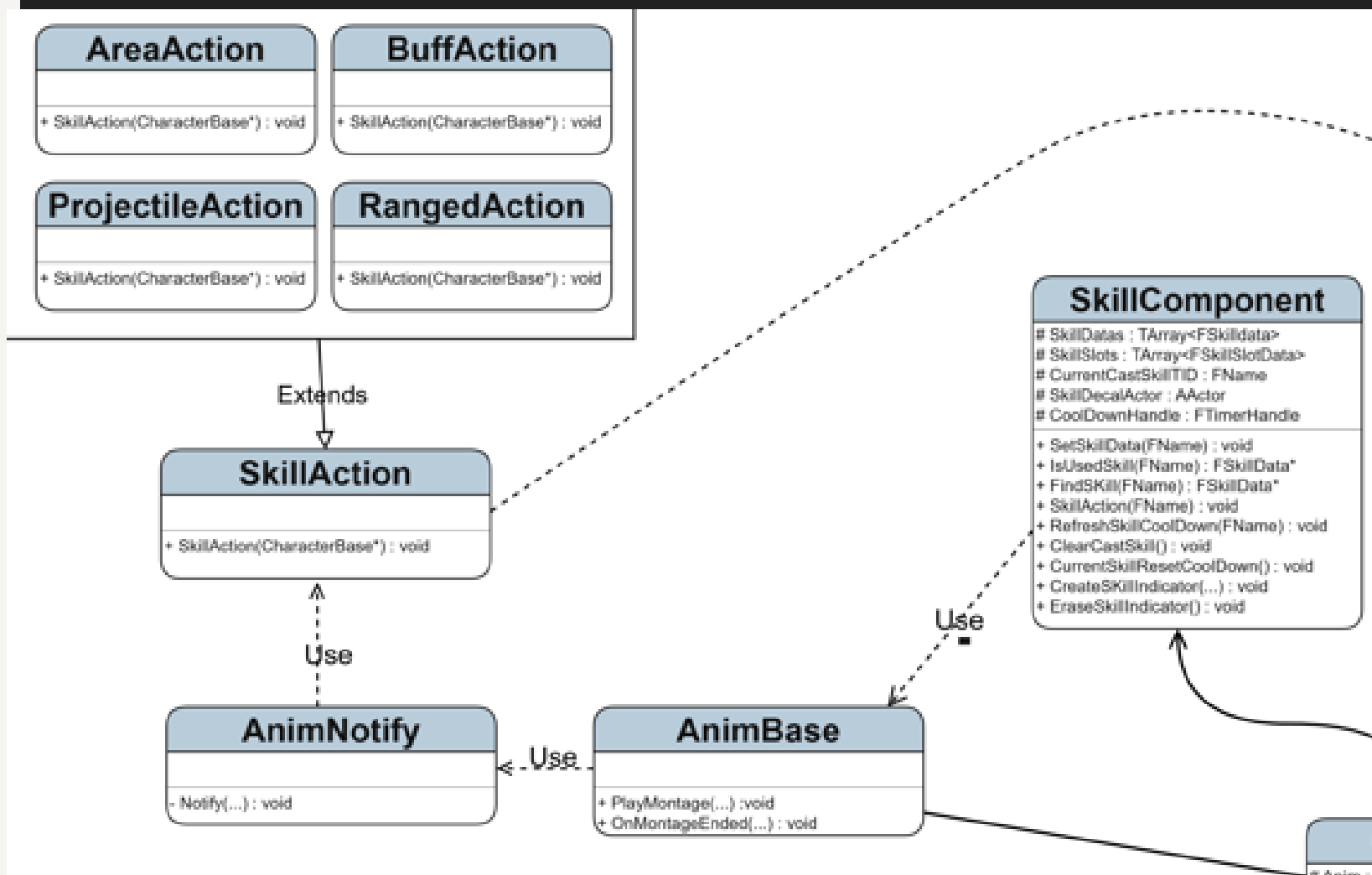
- IsLocallyControlled() 조건을 통해 자신이 직접 조종하는 캐릭터일 경우에만 서버로 스킬 실행 요청을 전송합니다.
- 이를 통해 클라이언트 애니메이션 타이밍에 맞춰 Server_ExecuteSkillAction()을 안전하게 역호출하며 스킬의 실제 효과는 서버 권한 하에서 일관되게 처리됩니다.



```
void USkillComponent::Server_ExecuteSkillAction_Implementation()
{
    ACC_CharacterBase* OwnerCharacter = Cast<ACC_CharacterBase>(GetOwner());
    if (!OwnerCharacter || !CurrentSkillTableData.SkillActionClass) return;

    // 1. 데이터 테이블의 클래스 타입 기반으로 스킬 객체 동적 생성
    USkillActionBase* ActionInstance =
        NewObject<USkillActionBase>(this, CurrentSkillTableData.SkillActionClass);

    if (ActionInstance)
    {
        // 2. 부모 인터페이스(가상 함수) 호출 -> 다형성 실행
        ActionInstance->SkillAction(OwnerCharacter);
    }
}
```



다형성 기반 스킬 로직 동적 실행

- 스킬 실행 로직을 SkillComponent 로 부터 분리하여 데이터 테이블에 정의된 액션 클래스 (SkillActionBase)를 기반으로 런타임에 스킬 로직을 동적으로 생성하고 실행하는 구조로 설계되었습니다.
- 스킬의 실제 동작은 공통 추상 클래스인 SkillActionBase에 의존하며 구체적인 구현은 이를 상속한 개별 액션 클래스에서 담당합니다.
이를 통해 다형성을 활용한 확장 가능하고 유연한 스킬 실행 구조를 구현하였습니다.



1-5 Ability System



Ability system은

캐릭터의 모든 능력치를 **출처별 데이터 컨테이너**로 분리 관리하고 상태 변화가 발생할 때 마다 원본 데이터를 직접 수정하지 않고 합산 결과를 재계산하는 **데이터 구동형 능력치 관리 시스템**입니다.

본 시스템은 장비, 버프, 디버프 등 각 능력치 변경 요소를 독립적인 컨테이너로 관리하여 새로운 요소 추가 시 기존 로직 수정 없이 확장이 가능하며 단방향 합산 구조를 통해 데이터 흐름이 명확해 유지보수가 용이하도록 설계되었습니다.



1) 어빌리티 시스템 컨테이너 구조

```
UPROPERTY(ReplicatedUsing = OnRep_DefaultAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> DefaultAbils;
```

```
UPROPERTY(ReplicatedUsing = OnRep_ItemEquipAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> ItemEquipAbils;
```

```
UPROPERTY(ReplicatedUsing = OnRep_ItemOwnedAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> OwnedAbils;
```

```
UPROPERTY(ReplicatedUsing = OnRep_BuffAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> BuffAbils;
```

```
UPROPERTY(ReplicatedUsing = OnRep_DeBuffAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> DeBuffAbils;
```

```
// 아이템, 버프 등 모든 효과가 합산된 최종 능력치 목록
```

```
UPROPERTY(ReplicatedUsing = OnRep_TotalAbils)
```

```
0 Blueprints에서 변경됨
```

```
TArray<FAbilityData> TotalAbils;
```

원본 데이터 분리 철학

- 본 Ability System은 기본 스탯(Base Stat)과 모디파이어(장비, 버프, 디버프 등)를 명확히 분리하여 능력치를 관리하는 구조로 설계되었습니다.
- 기본 스탯은 데이터 테이블에 정의된 원본 데이터로서 절대 직접 수정되지 않으며 장비, 버프, 디버프 등의 효과는 Default / Equip / Owned / Buff / Debuff와 같은 출처별 컨테이너 레이어로 분리되어 관리됩니다.
- 모든 효과는 합산 과정을 통해 최종 능력치(TotalAbils)로 계산되며 TotalAbils는 항상 계산 결과만을 보관합니다.

데이터 무결성 보장

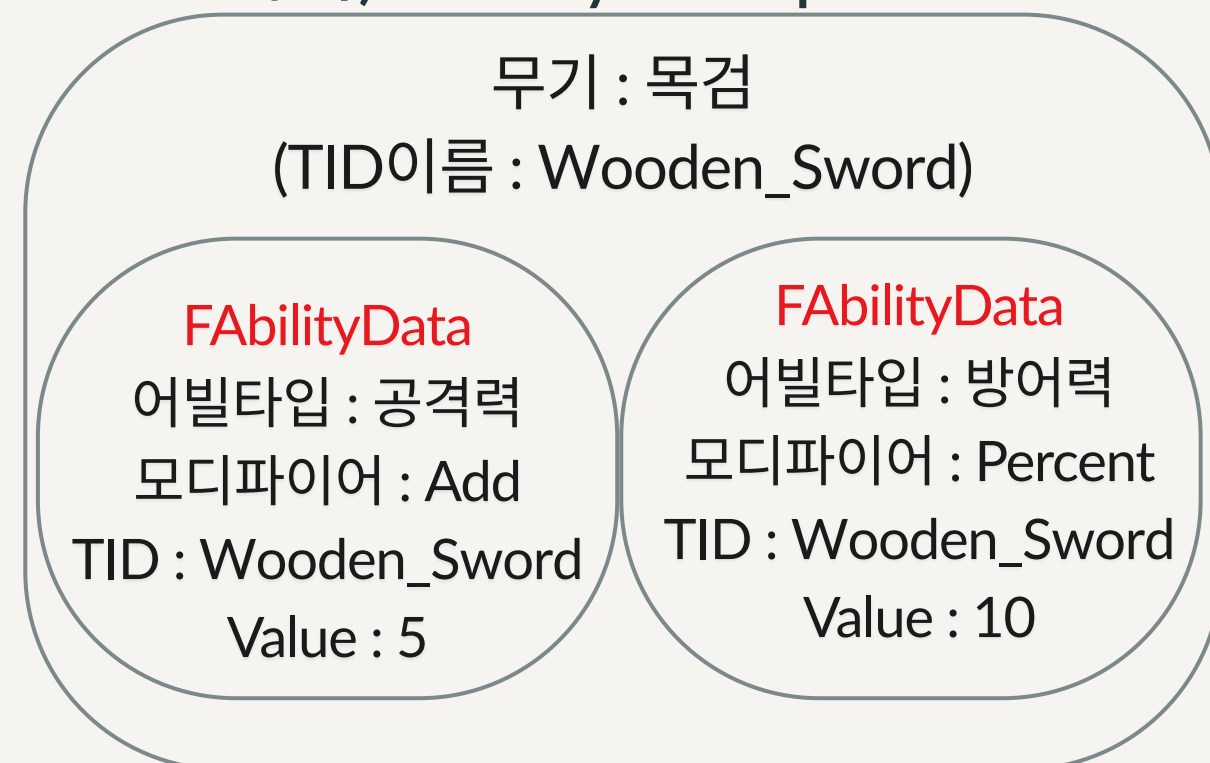
- 능력치 변경은 항상 재계산 'RefreshTotalAbil()' 을 통해 처리되므로 런타임중 원본 데이터가 오염되는것을 원천적으로 방지합니다.
- 이를 통해 버프 종료나 장비 해제 시 능력치가 정확하고 안전하게 원래 상태로 복원되는 구조를 보장합니다.



2) 어빌리티 시스템 데이터테이블 구조

구조 레벨	명칭	설명	비고
Level 1	어빌리티 데이터 (FAbilityData)	개별 스탯의 최소 단위 (예: 공격력 +10 , 방어력 +5%)	수치 및 연산 타입 포함
Level 2	어빌리티그룹테이블 AbilityGroupTable	특정 캐릭터, 아이템, 버프가 가지는 어빌리티 세트 (예: 캐릭터 기본 스탯 , 반지 능력 치 등)	Level 1 데이터들의 집합

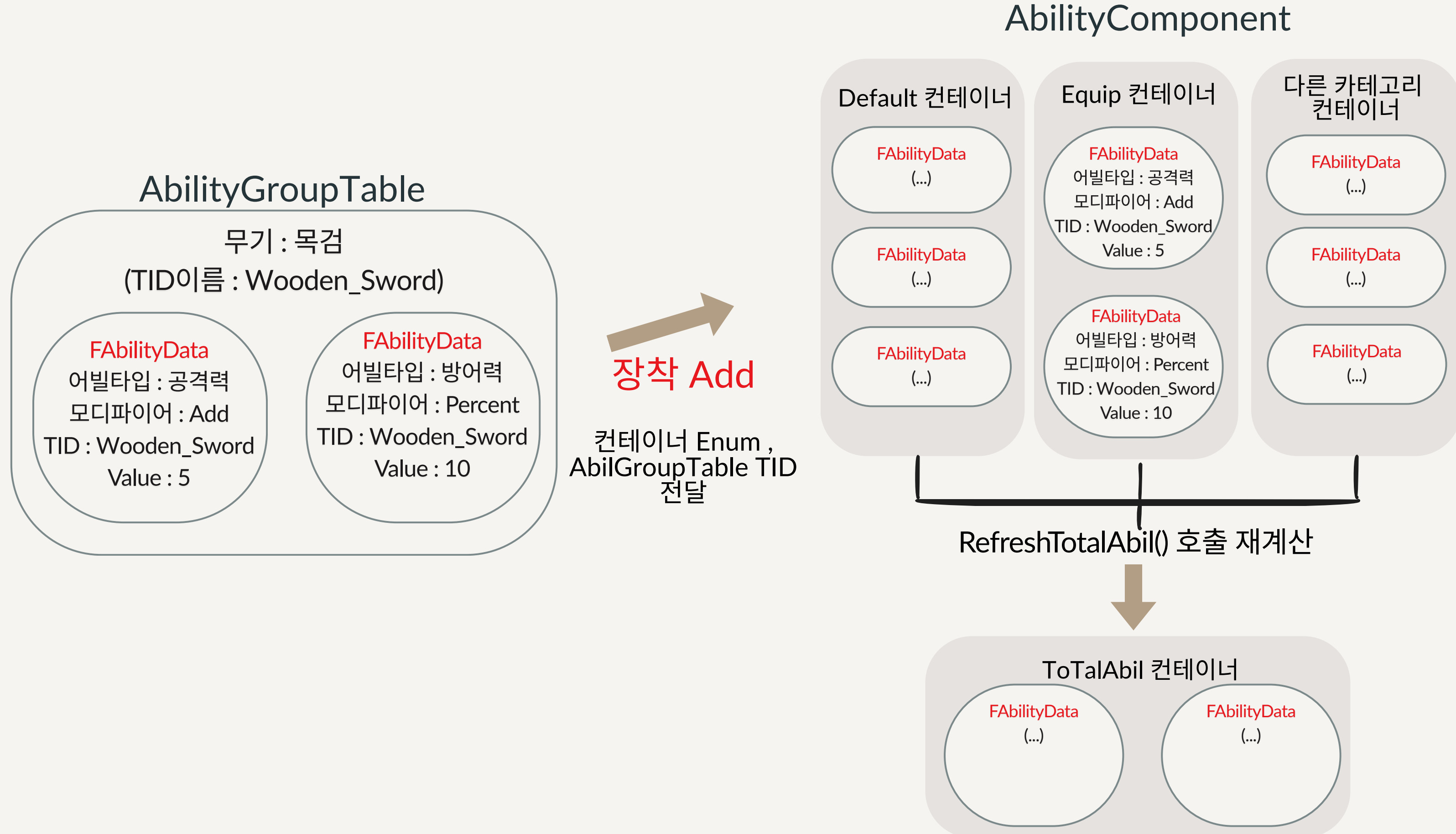
예시) AbilityGroupTable



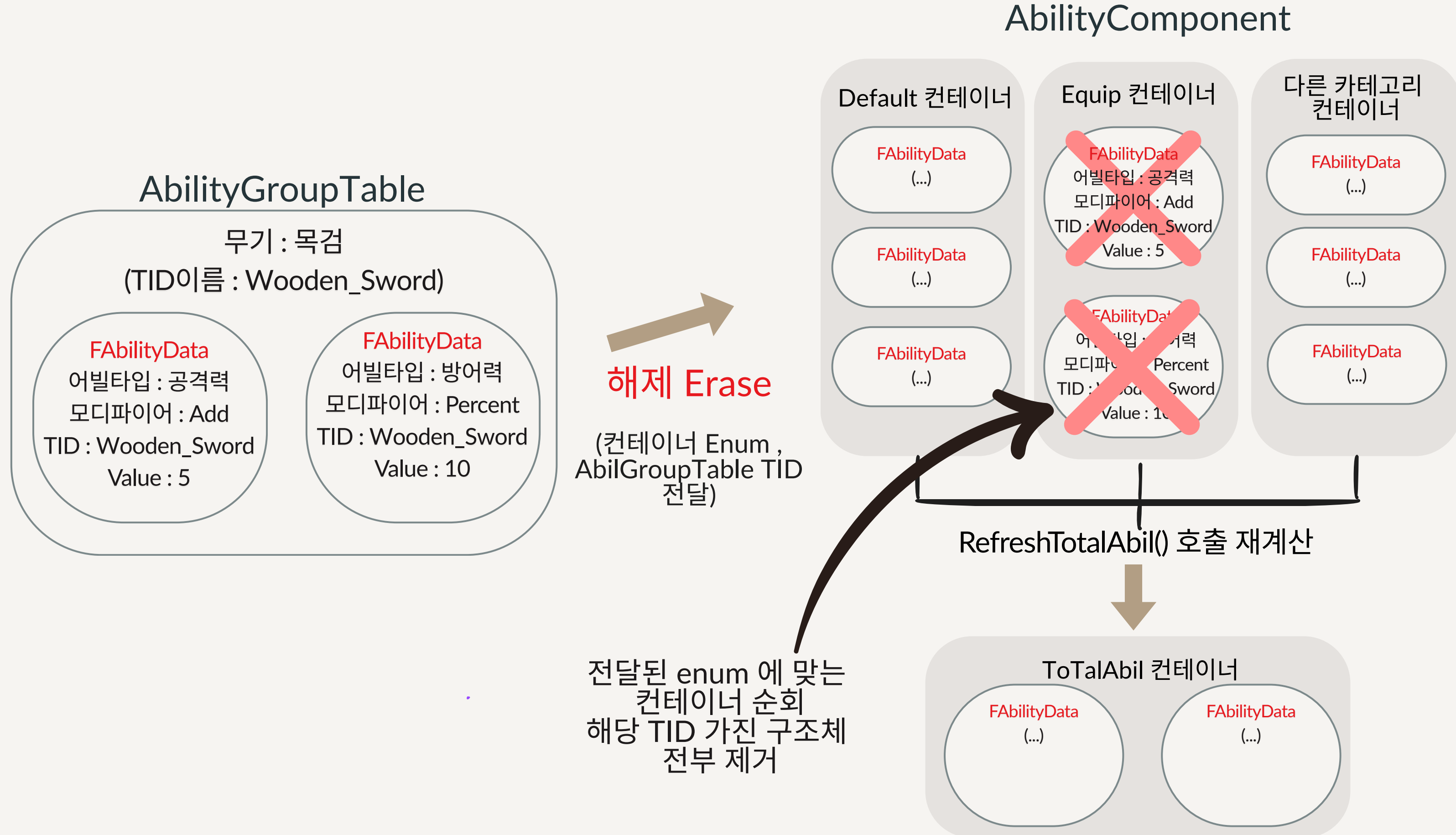
어빌리티 그룹 테이블 컨테이너에 포함된 각 AbilityData 구조체에는 자신이 속한 어빌리티 그룹 테이블의 TID를 동일하게 보유하도록 설계되었습니다.

이를 통해 그룹 테이블의 TID만을 기준으로 출처별 컨테이너에서 해당 그룹에 속한 어빌리티들을 일괄적으로 추가하거나 제거할 수 있는 구조를 제공합니다.

3) 어빌리티 시스템 작동 방식(스텝 추가)



3) 어빌리티 시스템 작동 방식(**스텝 삭제**)





4) 데이터 흐름 파이프라인 정리

[이벤트 발생] (디폴트 어빌 부여, 아이템 장착/버프 획득 등)



[카테고리 가감] (AddAbilData(Enum,TID) / EraseAbilData(Enum,TID))
호출로 해당 컨테이너만 업데이트



[최종 스탯 재계산] (RefreshTotalAbil 호출)

1. 고정치(float)합산
2. 퍼센트(Percent) 계수 반영 후 소수점
3. 디버프(Minus/Percent) 최종 차감 및 최소값 보정



[자원 및 UI 동기화] (UpdateCurrentHP_And_MP 호출)

1. 변경 전/후 Max 스탯 비율 계산하여 현재 HP/MP 비율 유지
2. OnHPChanged 델리게이트 브로드캐스트-> 클라이언트 UI 즉시 갱신

본 시스템은 원본 스탯 데이터를 직접 수정하지 않고,
상태 변화 시 출처별 컨테이너만 가감한 뒤 Refresh를 통해 최종 스탯을 재계산하는
단방향 데이터 흐름을 유지합니다.

구조적 강점 및 아키텍처적 이점

- 안정적인 유지보수

아이템 해제나 버프 만료 시, 전체 스탯을 역산하지 않고
해당 컨테이너에서 TID 제거 후 Refresh만 수행하여
연산 오류로 인한 스탯 꼬임을 구조적으로 방지합니다.

- 비율 기반 스탯 동기화

최대 HP 증가와 같은 스탯 변화 시 현재 값은 기존 비율을 기준으로 재조정되어,
의도치 않은 사망이나 오버플로우 없이 자연스럽게 동기화됩니다.

- 네트워크 최적화 및 보안

모든 스탯 계산과 타이머 처리는 서버에서만 수행되며,
클라이언트는 리플리케이션을 통해 UI 갱신만 담당합니다



1-6 Item & Inventory



본 시스템은 Inventory와 Equipment의 책임을 명확히 분리한 구조로 설계되었습니다.

- Inventory는 아이템의 보관과 수량 관리에만 집중하며
- Equipment는 실제 캐릭터 상태 변화와 능력치 적용을 전담합니다.

특히 아이템을 보관할 때 아이템 액터를 생성하여 유지하는 방식이 아닌 **데이터 테이블의 TID를 기반으로 한 경량 구조체(FItemData) 형태로만 관리함**으로써 불필요한 액터 생성 및 메모리 점유를 최소화하고 대규모 인벤토리 환경에서도 안정적인 성능을 유지할 수 있도록 설계되었습니다.



[InventoryComponent]

```
UPROPERTY(EditAnywhere, BlueprintReadWrite, Replicated)
0 Blueprints에서 변경됨
TArray<FItemData> InventoryItems;
```

```
struct FItemData
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int64 UID;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    0 Blueprints에서 변경됨
    FName TID;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    0 Blueprints에서 변경됨
    int32 Count = 0;
```

아이템은 인벤토리에 보관되는 순간

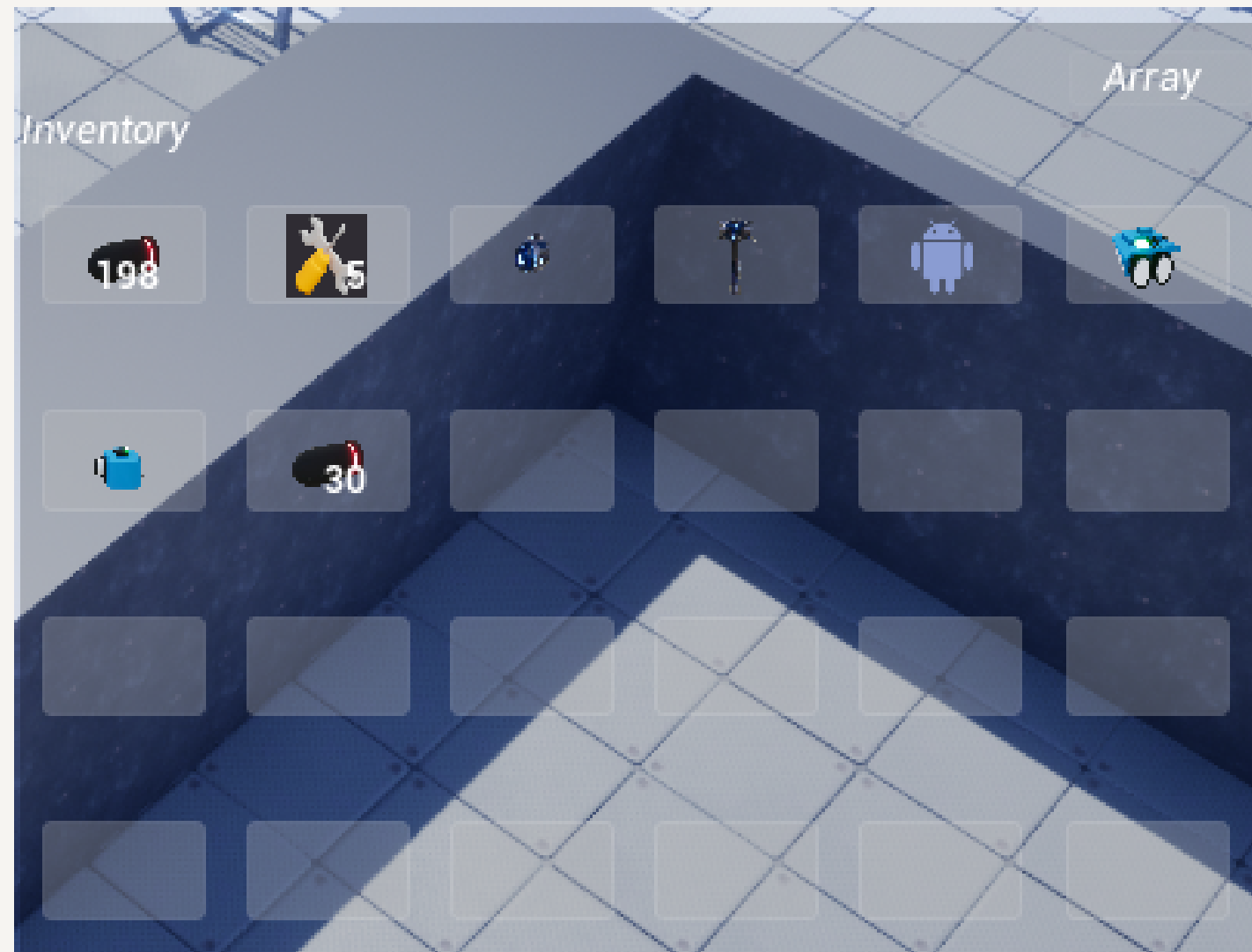
월드 액터 형태가 아닌 TID 기반의 경량 구조체로 변환되어 관리됩니다

이 구조를 통해 인벤토리는

아이템의 시각적 표현이나 로직을 전혀 보유하지 않으며

오직 아이템 식별 정보와 최소 상태 데이터만을 컨테이너에 저장합니다.

그 결과 다량의 아이템을 보관하더라도 불필요한 액터 생성, 메모리 점유, Tick 비용을 제거할 수 있으며 아이템 액터는 실제로 필요해지는 시점에만 생성되는 효율적인 라이프사이클을 유지합니다.



```
struct FItemTableRow : public FTableRowBase
{
    GENERATED_BODY()
    // UI에서 보여질 이미지텍스처
    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    UTexture2D* Image;
```

인벤토리를 열거나 아이템 목록을 표시할때 아이템 액터를 생성하지 않습니다.

인벤토리에는 FItemTableRow 기반의 경량 데이터(TID) 만을 보관하며 UI에서 는 해당 데이터 테이블에 정의된 Image(Texture) 정보를 참조하여 아이콘을 그리는 방식으로 처리됩니다.

이를 통해 UI표시를 위해 불필요한 액터 생성 비용을 제거하고 인벤토리 열기/닫기 시에도 메모리 사용량과 성능 오버헤드를 최소화했습니다.



```
struct FItemTableRow : public FTableRowBase
{
    GENERATED_BODY()
    // 아이템 메시
    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    UStaticMesh* Mesh;..
```

[InventoryComponent]
(TArray<FItemData>)



Equip(ItemTID)

[EquipmentComponent]
(TArray<FItemEquipSlotData>)



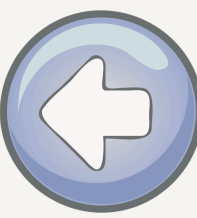
시각적 표현

[Character Mesh]
(Slot / SkeletalMesh 교체)

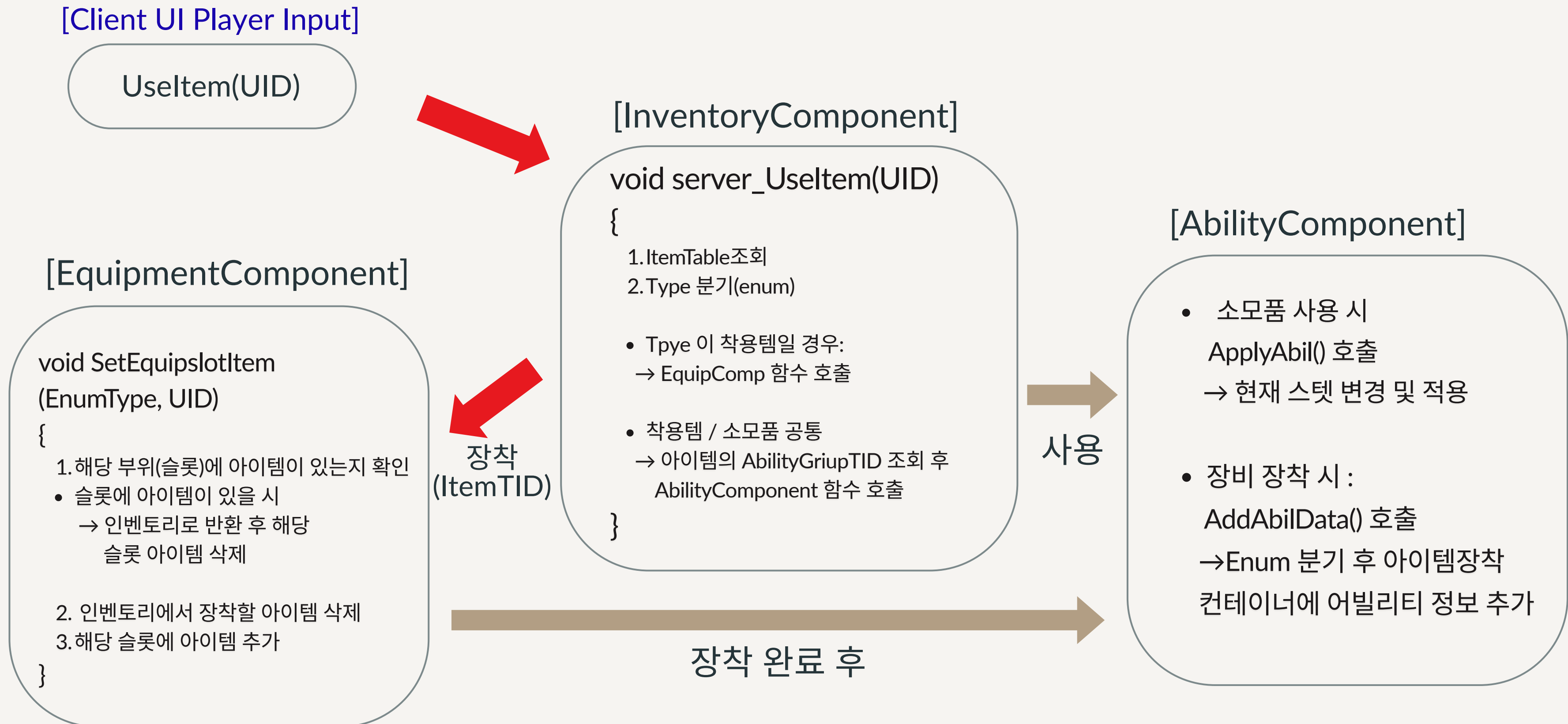
아이템 장착 과정에서도 액터를 생성하지 않으며
Inventory에서 전달된 TID 기반 데이터만을
EquipmentComponent에 반영합니다.

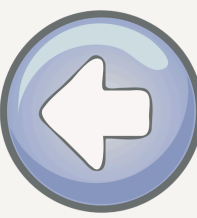
이후 장착 결과는 능력치와 메쉬 변경으로 분리되어 처리됩니다.

액터 생성은 월드 상 상호작용이 필요한 경우로만 제한했습니다.



Item 착용 및 사용 시 데이터 기반 처리 흐름





Item **해제** 시 데이터 기반 처리 흐름

[Client UI Player Input]

UseItem(UID)



[EquipmentComponent]

```
void server_UnEquipItem(SlotType)
{
    해당 부위(슬롯)에 아이템이 있는지 확인

    • 슬롯에 아이템이 있을 시
      → 인벤토리 컴포넌트의
        TryAddItem(FItemData, AItemBase)호출
      → 어빌리티 컴포넌트의
        EraseAbilData(Category, TID) 호출
}
```

해제
(ItemTID)

[InventoryComponent]

```
void server_TryAddItem
(FItemData, AItemBase)
{
    1. 해당 아이템 맥스 스택 카운트 확인
    2. 획득해야 할 아이템 수량 확인
    3. 장착 아이템은 보통 1이기에 인벤토리 공
       간 여분이 있으면 해당 아이템 추가 완료
}
```

[AbilityComponent]

- 장비 해제 시 :
EraseAbilData() 호출
→ Enum 분기 후 아이템장착
컨테이너에 어빌리티 정보 삭제

해제 완료 후



시스템 설계 원칙 요약

구분	진입점	핵심 처리	상태 변경 주체
장착	InventoryComponent	아이템 소유 검증 후 장착 요청 전달	EquipmentComponent
사용	InventoryComponent	사용 가능 여부 검 증 후 효과 실행	AbilityComponent
해제	EquipmentComponent	장착 상태 제거 및 효과 정리	Equipment->Ability ->Inventory

1. 서버 권한 준수

- 모든 상태 변경은 해당 상태를 소유한 컴포넌트에서만 수행하도록 설계하였습니다.
Inventory 는 소유 상태만을 관리하고
Equipment는 장착 상태만을 관리함으로써
서버 권한 기반의 단일 책임 원칙을 유지합니다.

2. 데이터 단방향 유지

- 아이템 장착 및 사용은
소유 → 상태 → 효과의 단방향 흐름으로 처리되며
- 해제 시에는
상태 → 효과제거 → 소유복구 순서를 따릅니다.

3. 책임 분리 및 확장성

- 각 컴포넌트는 자신의 책임 범위 내에서만 동작하며
새로운 아이템 유형이나 효과가 추가되더라도
기존 구조를 수정하지 않고 확장이 가능한 구조를 유지합니다.



아이템 월드 스폰 흐름도

Monstser

컨테이너

[Item Drop List]

ItemTID

드롭 확률

최대 드롭 수량

[Item Drop List]

ItemTID

드롭 확률

최대 드롭 수량



확률 판정



ItemTID 조회



[FItemTableRow]

아이템 클래스 조회
(ItemBase)



1. SpawnActor<ItemBase>
2. Initialize(TID, UID, count)

[World Item Actor]

몬스터는 드롭 결정자 역할만 담당하며
아이템의 실제 속성은
Item Table + ItemBase 초기화 로직에서
분리 관리됩니다.

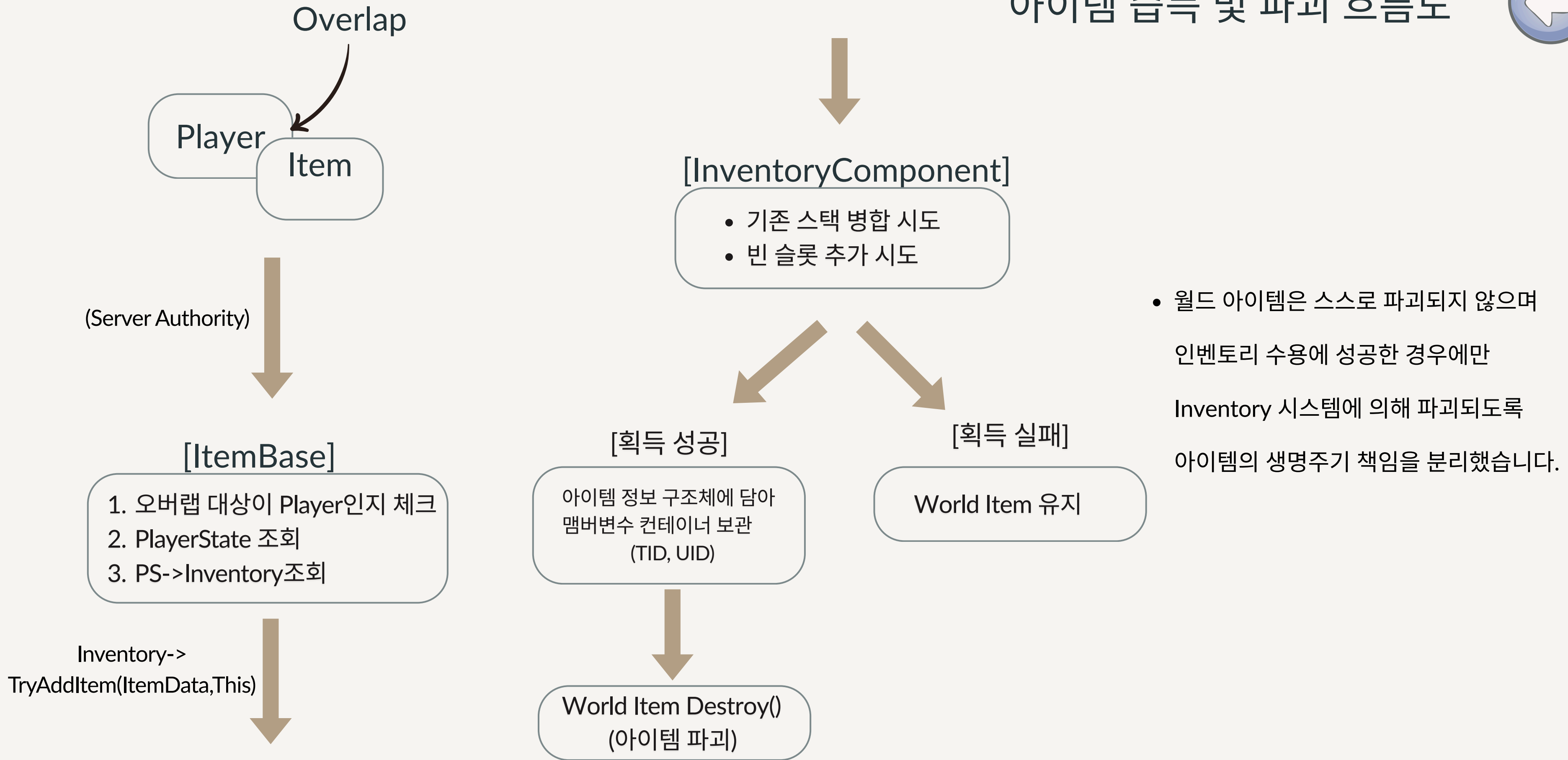


▼ Item			
▼ Item Drop List		배열 엘리먼트: 2	⊕ ⊖ ↶
▼ 인덱스 [0]		멤버: 3	▼ ↶
	Item Dropchance	70.0	↶
	Item TID	Ammo	↶
	Max Drop Count	30	↶
▼ 인덱스 [1]		멤버: 3	▼ ↶
	Item Dropchance	100.0	↶
	Item TID	Potion_HP_01	↶
	Max Drop Count	3	↶

몬스터 액터는

ItemDropList 라는 배열 데이터를 통해
드롭정보를 관리합니다.

- Item Drop Chance : 해당 아이템이 드롭될 확률(0~100%)
- Item TID : 실제 아이템 데이터를 조회하기 위한 테이블ID
- Max Drop Count : 드롭 최대 수량





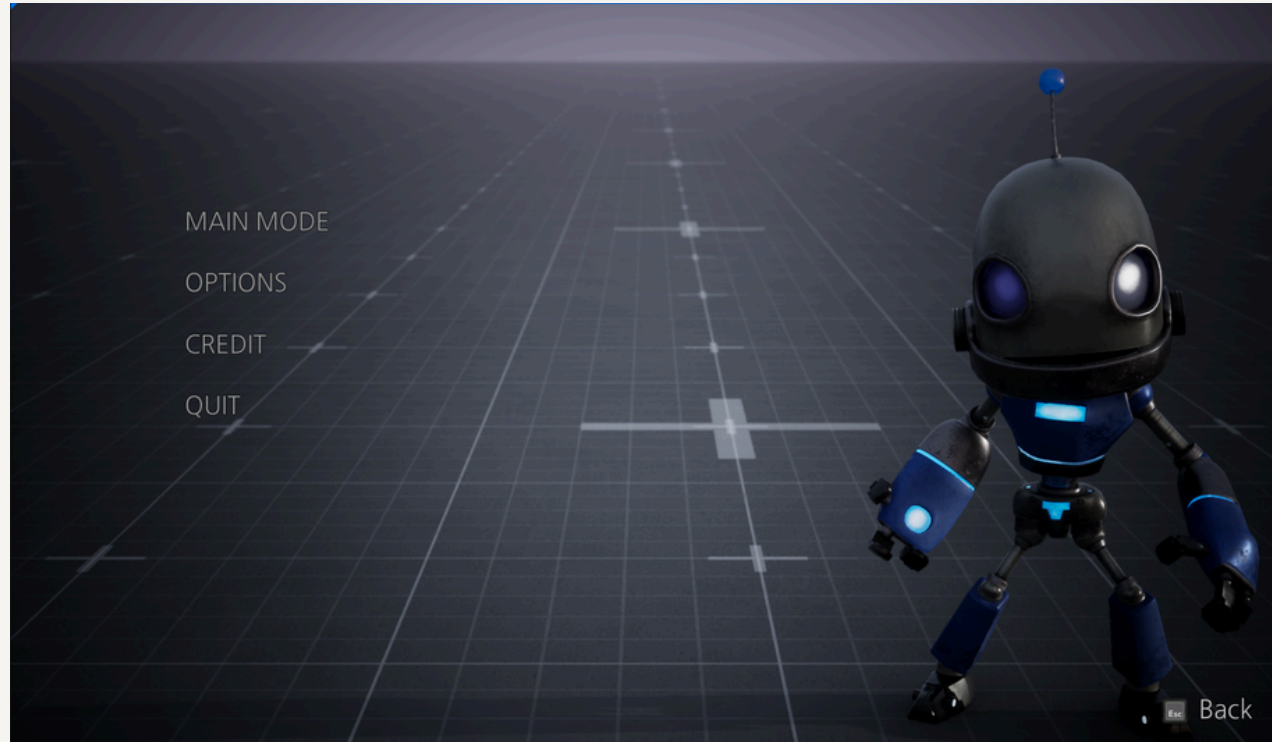
2-1 레벨 구성



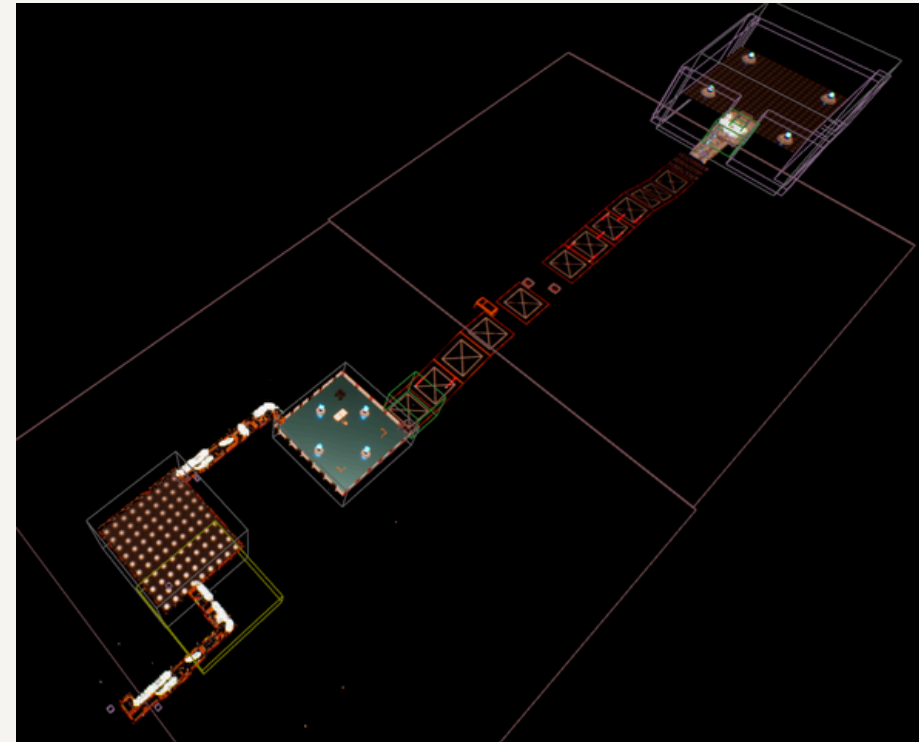
본 프로젝트의 레벨 구조는 로비를 중심으로 전투맵을 순환하는 액션 게임 흐름을 기반으로 설계되었습니다.

로비는 플레이어의 시작과 종료, 상태 정리 및 다음 전투로의 진입을 담당하는 허브 역할을 수행하며

각 전투 맵은 독립적인 플레이 경험을 제공한 뒤 다시 로비로 복귀하는 구조를 가집니다.



첫화면



스테이지



로비

본 게임은 크게 3가지의 맵 구조로 구성되어 있습니다

- 첫 화면 : 게임 모드 선택 및 각종 옵션을 설정하는 초기 진입 화면입니다.
- 로비 : 이동할 맵을 선택 등 주로 NPC와 상호작용을 중심의 공간입니다.
- 스테이지 : 게임의 핵심 전투와 주요 콘텐츠가 진행되는 공간입니다.



2-2 Widget

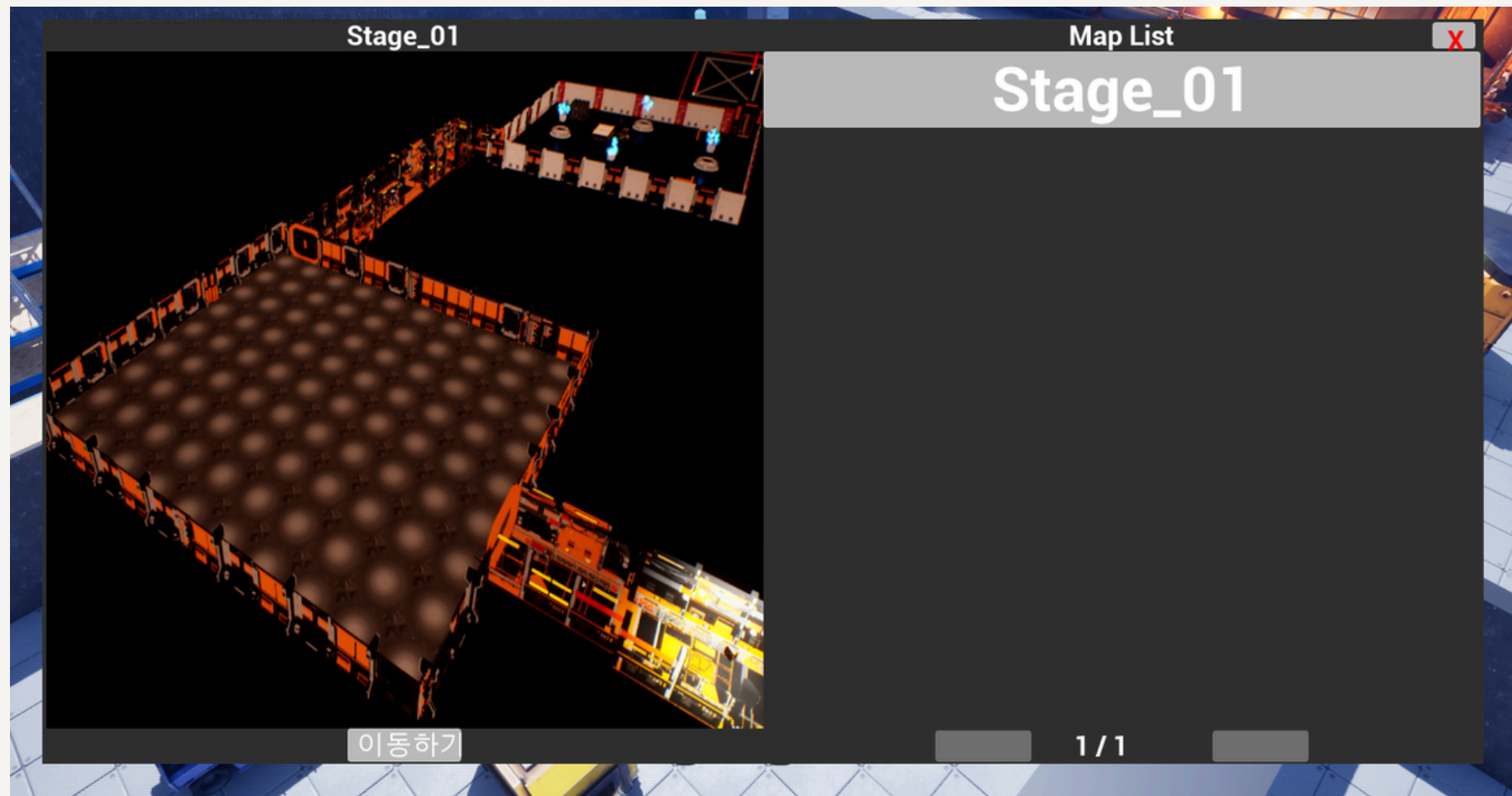


본 단원에서는

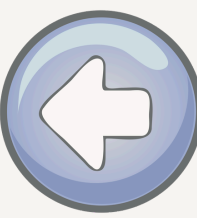
인게임에서 사용자에게 표시되는 위젯 요소의 구성과 역할을
설명합니다.



NPC와 상호작용



각 NPC는 데이터 테이블(NPC Table)을 통해 자신이 제공하는 기능과 UI를 데이터 기반으로 정의합니다. NPC는 단순히 상호작용 대상이 아니라, 어떤 위젯을 열 수 있는지, 어떤 흐름으로 UI를 시작할지에 대한 정보를 스스로 가지고 있습니다



NPC_Lobby ▾	
Type	NPC_Map ▾
NPCClass	BP_CC_NPCCharacter ▾
Name	MapNPC
Dialogue ID	
▼ NPCTIDDatas	
Map Group TID	Map_01
Item Shop TID	None
Tag TID	3
Animation TID	NPC_Map
▼ Widget Entry	배열 엘리먼트: 1
▼ 인덱스 [0]	멤버: 2 ▾
Widget Name	UI.Map X ▾
Widget Activatable	WBP_MapList ▾
Start Widget Tag	UI.Map X ▾

- NPC 테이블 하단의 Widget Entry는 해당 NPC와 상호작용 시 열 수 있는 UserWidget들을 의미하며 각 위젯은 Gameplay Tag를 기준으로 분류,관리됩니다.
- Start Widget Tag는 NPC와 상호작용했을 때 가장 먼저 표시될 위젯을 지정하며 Widget Entry 중 태그가 일치하는 위젯이 초기 화면으로 선택됩니다.
- 초기화 시 NPC는 자신이 가진 위젯 정보를 GameplayTag → Widget TMap(컨테이너) 구조로 저장하여 UI 조회 및 전환을 일관되게 처리합니다.

이를 통해 NPC별 UI 확장과 흐름 제어를 코드 수정 없이 데이터만으로 유연하게 구성할 수 있습니다.

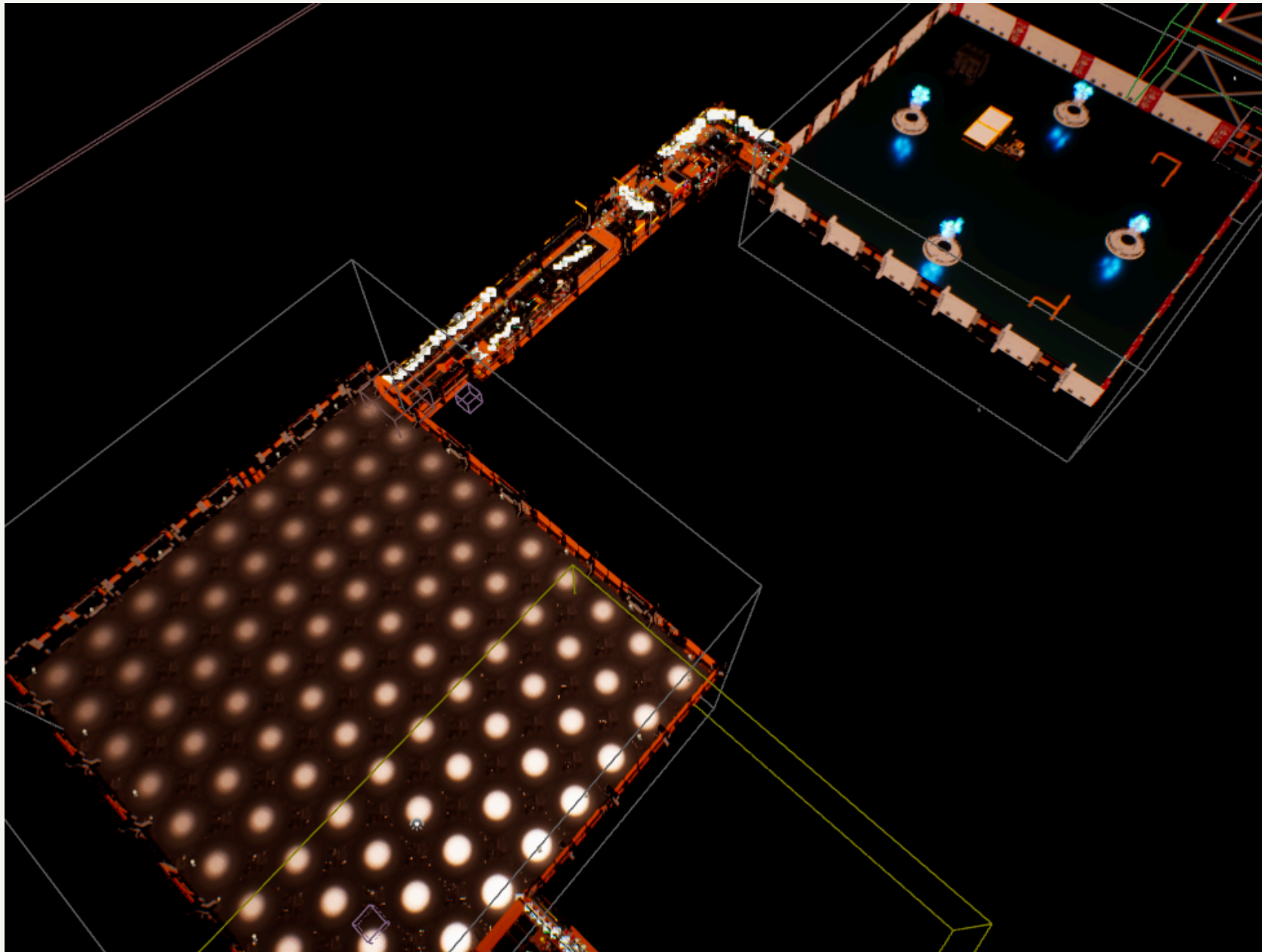


2-3 룸 상태 관리



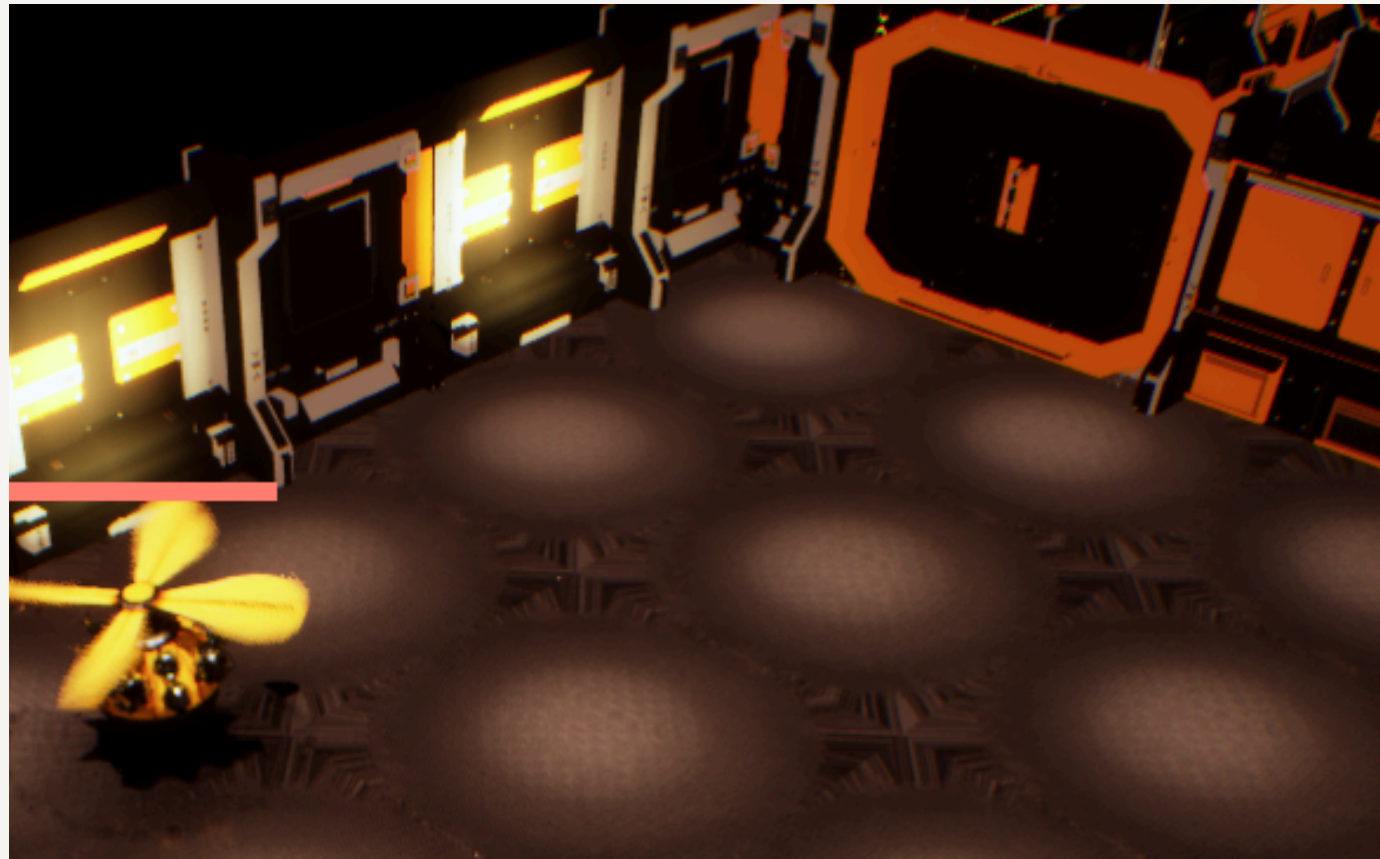
하나의 스테이지를 구성하는 개별 룸(Room) 단위의 콘텐츠 흐름과 이를 제어하는 구조에 대해 설명합니다.

플레이어가 방에 진입한 순간부터 몬스터를 처치하고 문과 상호작용하여 다음 구역으로 넘어가는 일련의 게임 플레이 시퀀스를 다룹니다.

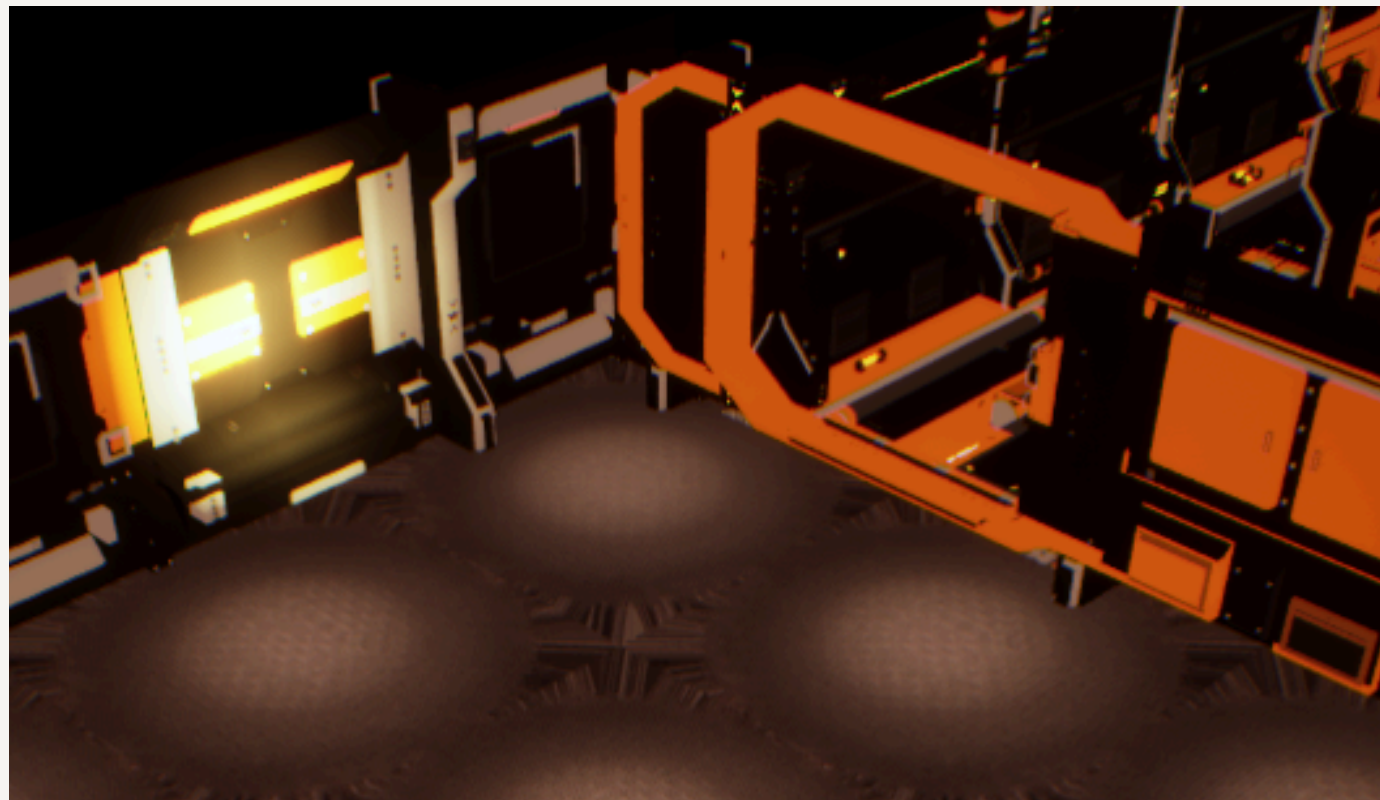


스태이지 01 내부 일부 방 모습

하나의 스테이지를 독립된 룸(Room)단위로 분할하여 구성하였으며 각 룸에 배치된 콘텐츠의 클리어 여부에 따라 전체적인 게임플레이가 순차적으로 진행되는 구조를 가집니다.



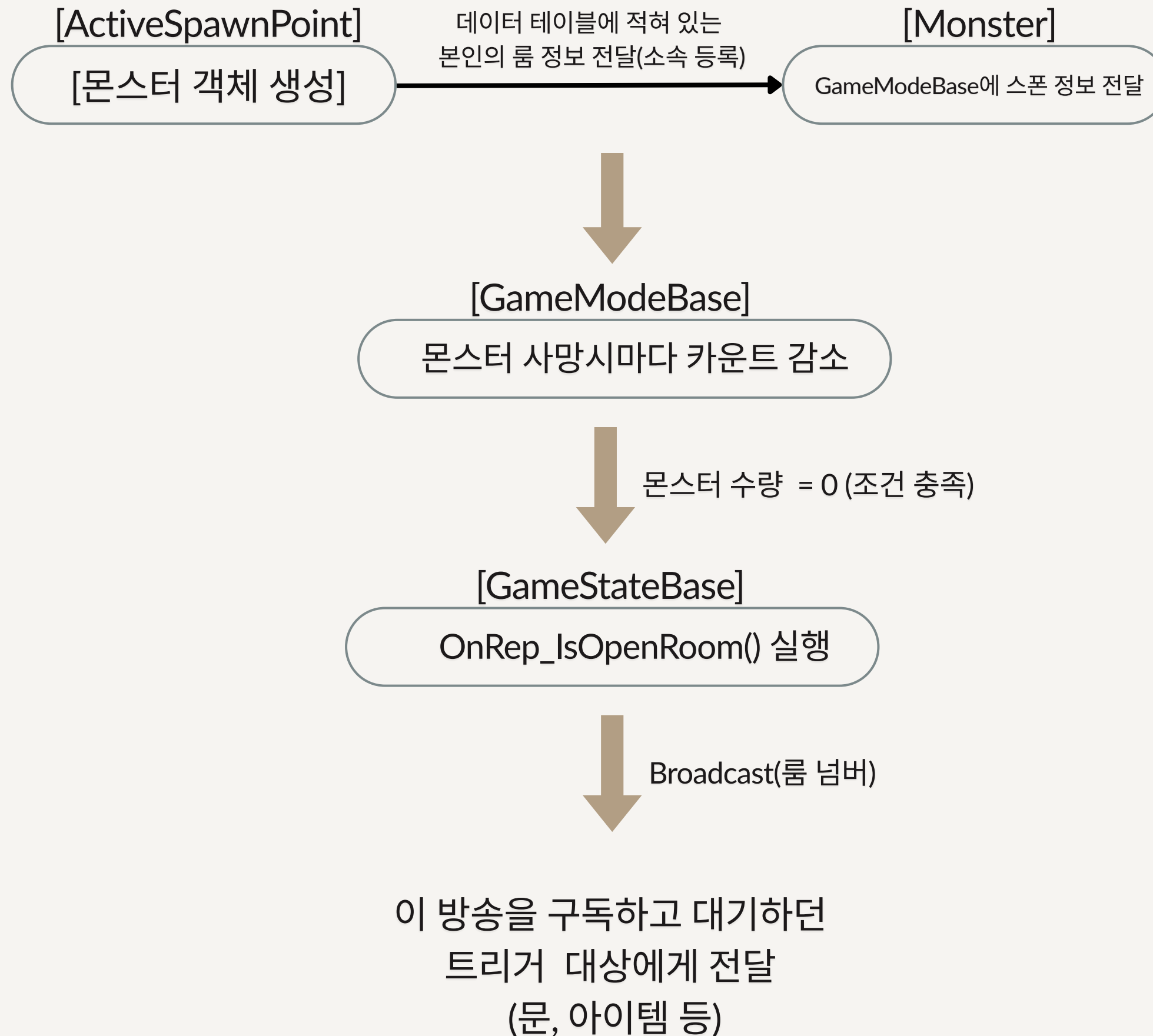
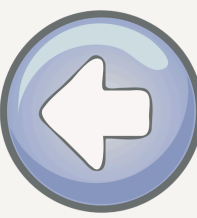
방 내부 몬스터 처리 전



방 내부 몬스터 처리 후 트리거 작동

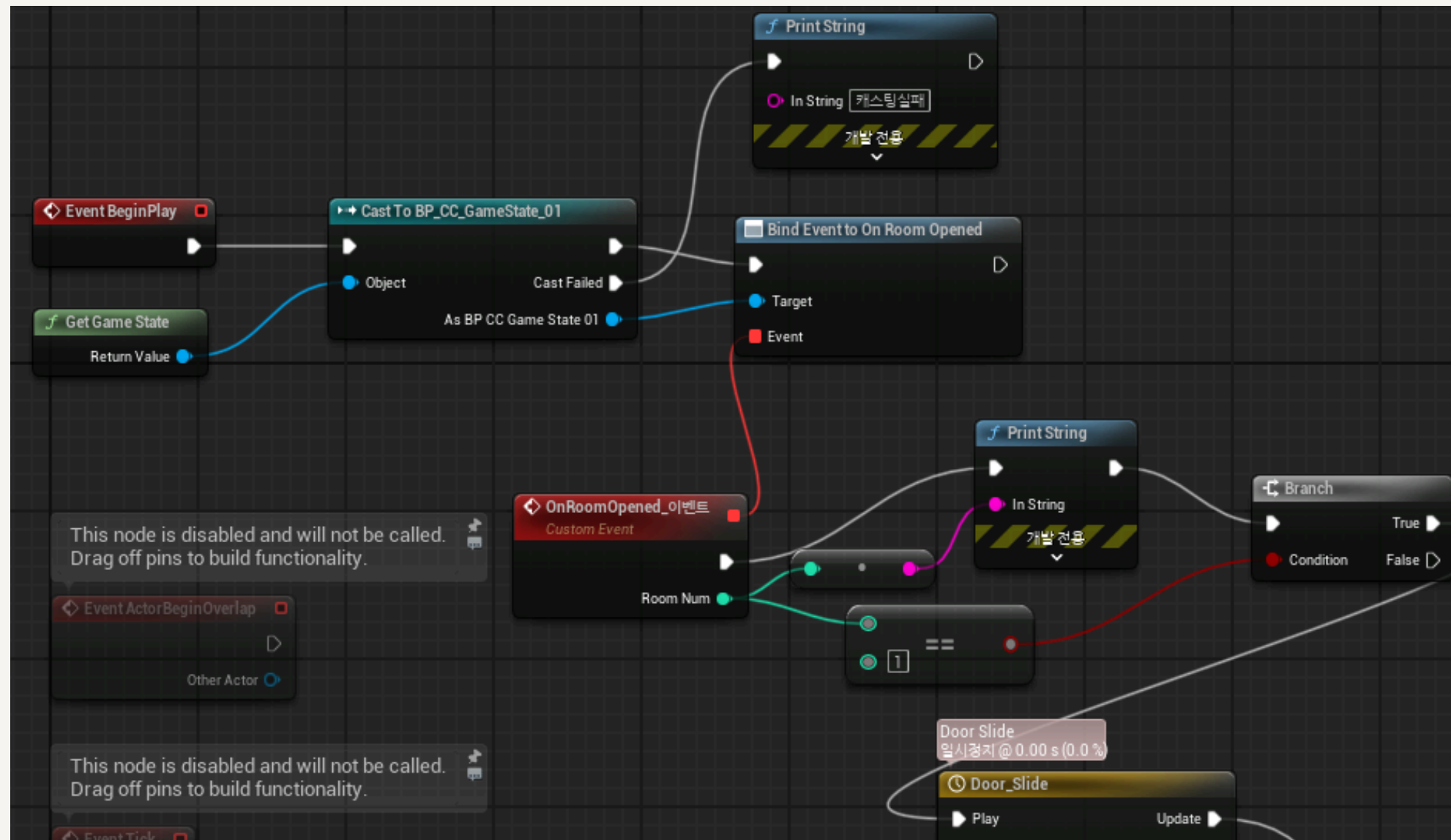
각 몬스터는 스폰과 동시에 GameModeBase에 본인이 속한 룸 (Room) 정보를 전달합니다.

이후 GameModeBase는 해당 구역 내의 몬스터 잔여 수량을 실시간으로 체크하며, 모든 몬스터가 제거되는 시점에 맞춰 해당 방의 클리어 트리거를 작동시킵니다.



개별 룸(Room)의 클리어 및 오브젝트 상호작용은 몬스터의 생성/소멸 이벤트와 네트워크 복제 메커니즘을 연동하여 제어합니다.
전체적인 흐름은 다음과 같습니다.

- 몬스터 스폰 시 : 몬스터가 생성되면 서버 GameModeBase에 소속 룸 정보를 등록하고 해당 룸의 활성화 몬스터 카운트를 증가(++) 시킵니다.
- 몬스터 사망 시 : 몬스터가 처치되면 카운트 감소(--) 시키며 카운트가 0이 되면 GameModeBase는 GameStateBase의 복제 변수를 클리어 상태로 갱신합니다.
- 클라이언트 동기화 및 방송 : 변수 값이 복제되면 각 클라이언트의 복제 콜백 함수(On_Rep)가 실행되며 해당 룸의 클리어 이벤트를 델리게이트로 브로드캐스트 합니다.
- 환경 오브젝트 반응 : 이 방송을 구독하고 대기하던 출입문 및 상호작용 물체들이 트리거를 수신하여 문 개방 등의 구역 시퀀스를 실행합니다.



본 로직은 GameStateBase의 방 개방 방송을 수신하여 해당 구역의 출입문을 자동으로 개방하는 클라이언트 반응 시스템입니다.

- 초기화 및 이벤트 구독(BeginPlay) : Get Game State 후 GameState로 형 변환 합니다. 성공 시 다중 캐스트 델리게이트이니 On Room Opened에 커스텀 이벤트를 바인딩하여 방송 구독을 완료합니다.
- 이벤트 수신 및 조건검사(Custom Event) : 방 개방 방송이 브로드캐스트 되면 파라미터로 방 번호를 수신합니다. 이후 비교 연산자(==)를 통해 본 액터가 제어 할 방 번호가 맞는지 판별합니다.
- 행동 실행(Branch&Timeline) 조건문 결과가 True일 경우 문이 부드럽게 열리는 타임라인을 실행하여 구역 개방 시퀀스를 완료합니다

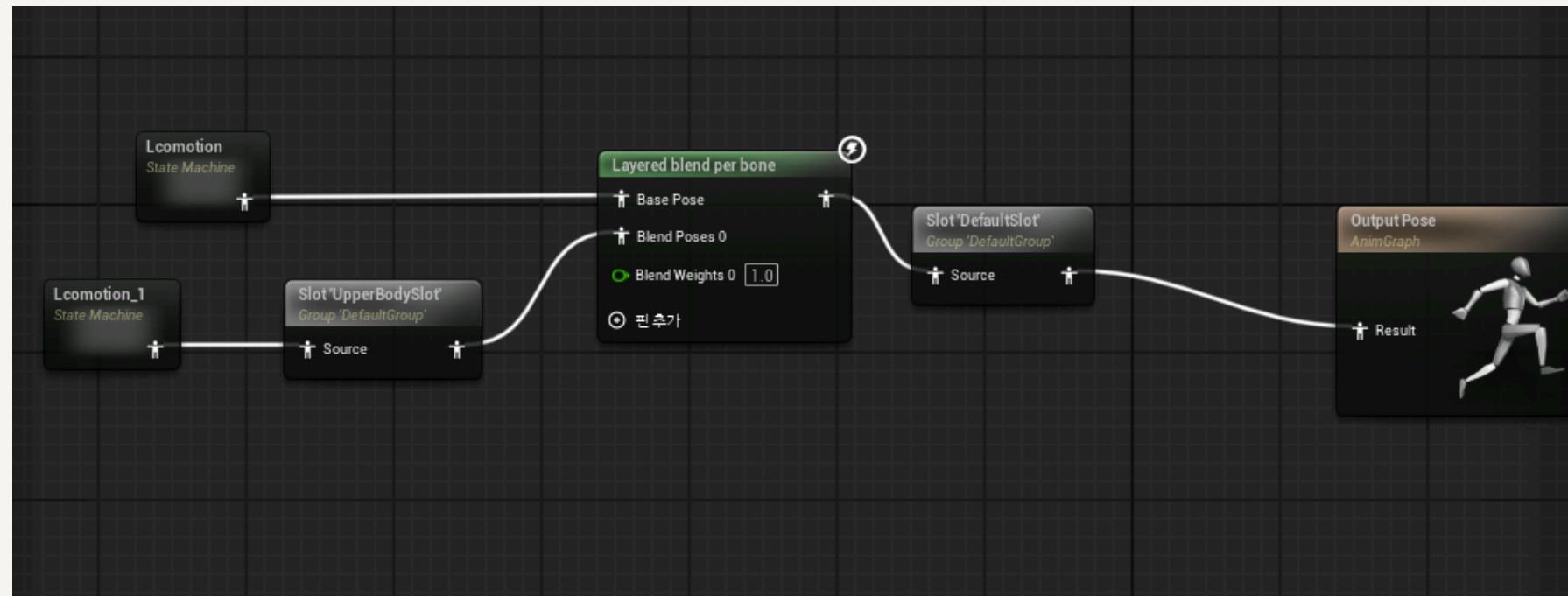


2-4 캐릭터 (플레이어 & 몬스터)



본 단원은 캐릭터의 자연스러운 움직임을 위한 상하체 분리로직과
비헤이비어 트리(BT) 및 테스크를 활용하여
유기적으로 변화하는 동적 시야 및 사거리 시스템을 설명합니다.

상하체 분리 로직 구현



애니메이션 블루프린트 로직

스켈레탈 메시의 본마스크를 활용해
상체와 하체의 애니메이션 영역을 분리하고
애니메이션 블루프린트 내에서
Layered blend per bone 노드를 통해
두개의 서로 다른 애니메이션 흐름을
유기적으로 합성했습니다.



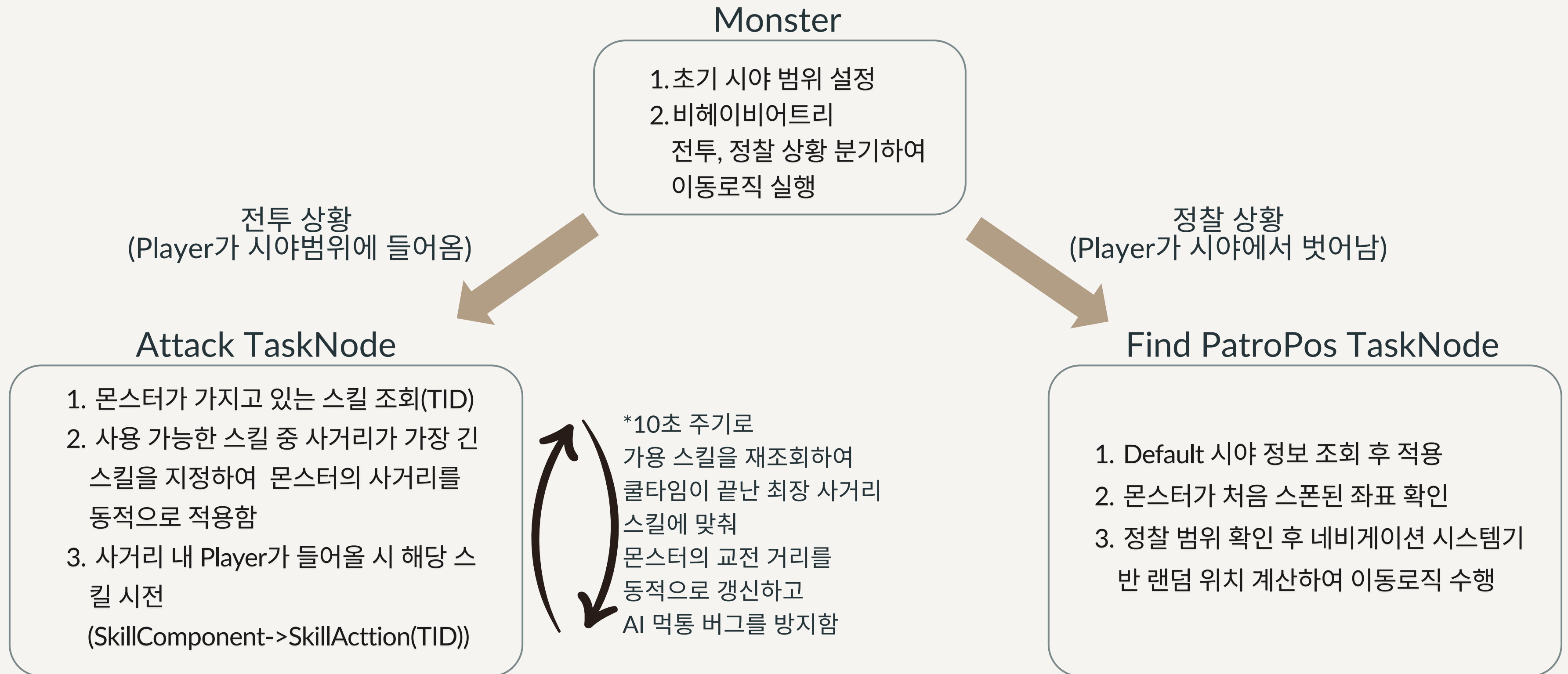
기존 달리기 모션



달리기 모션과 장전 모션이 같이 재생되는 모습

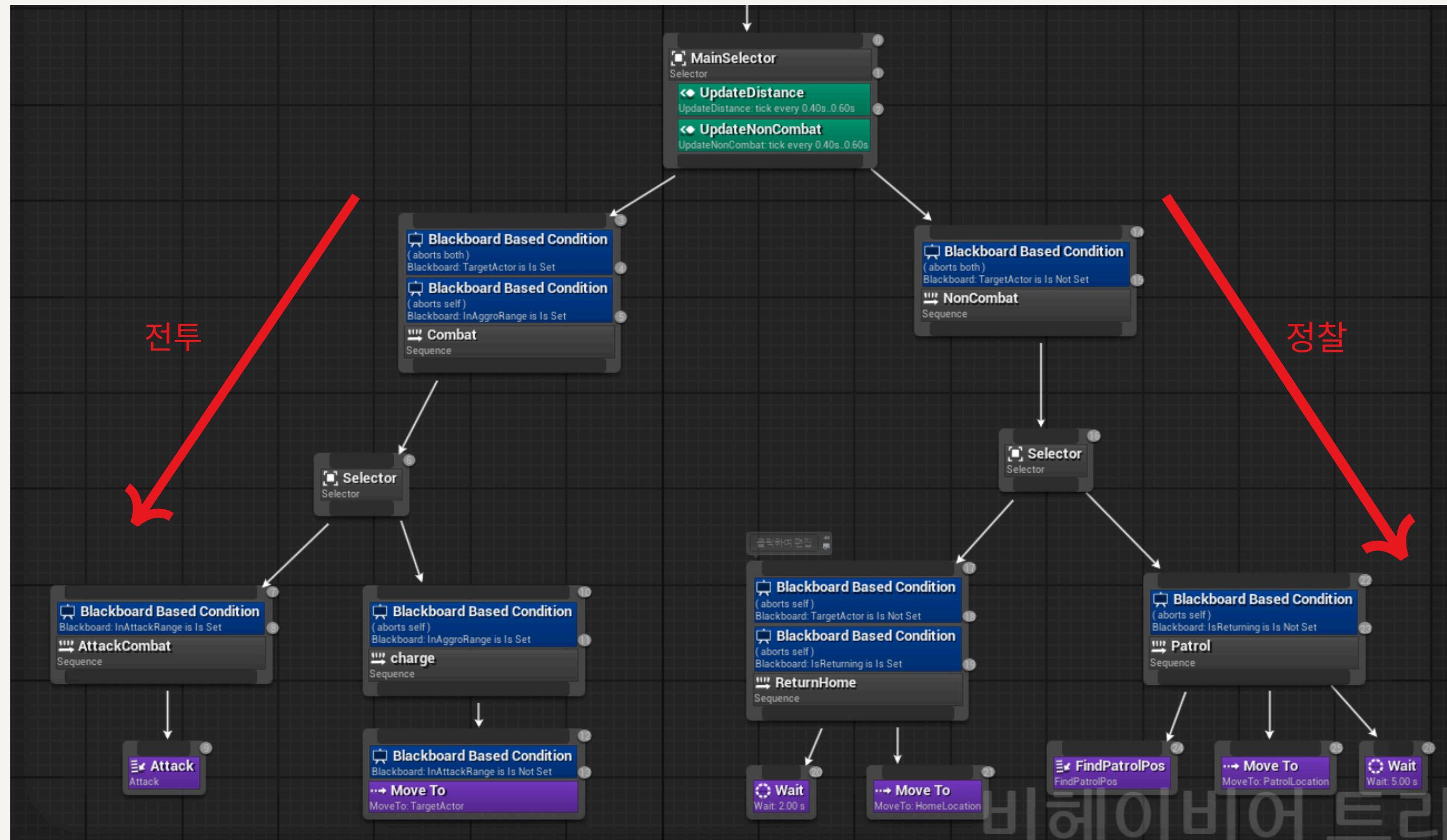


데이터 테이블 연동 기반 동적 AI이그로 및 스킬 시스템 흐름도





몬스터 비헤이비어 트리



전투와 정찰 로직을
분기하여 처리하였습니다.



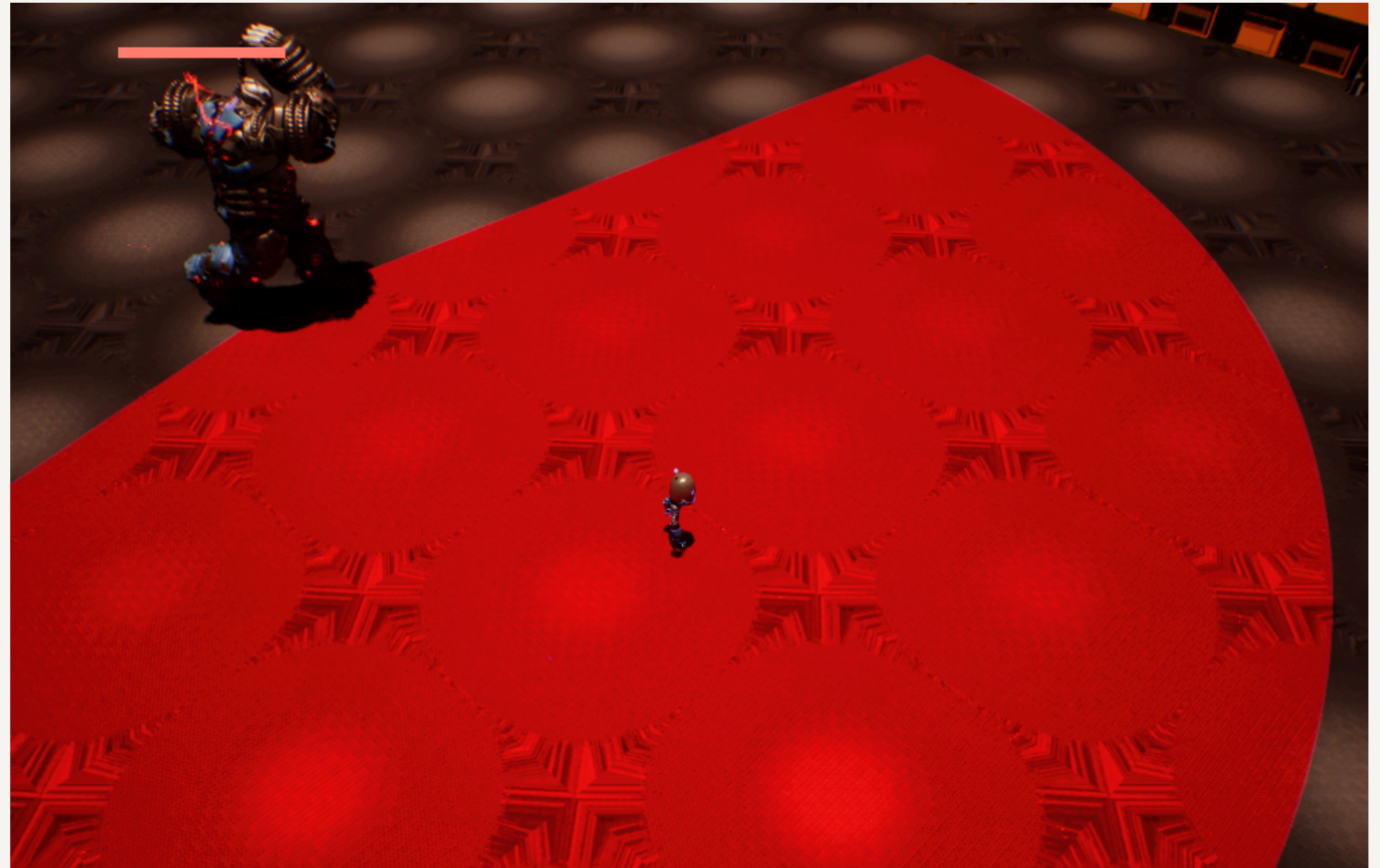
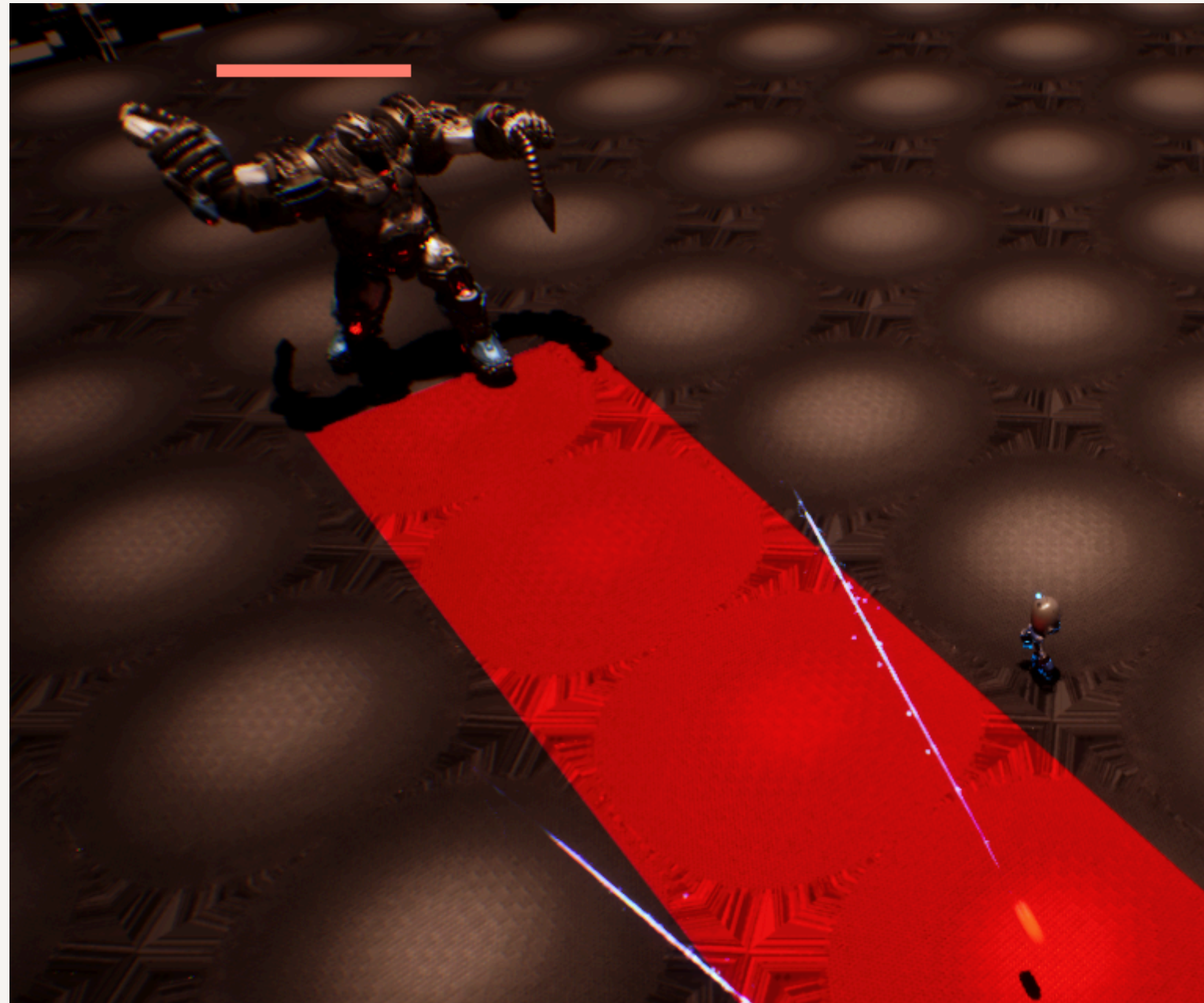
2-5 스킬인디케이트



전투 중 몬스터가 사용하는 스킬은 피격 범위와 형태를 사전에 인지할 수 있어야 공정한 플레이 경험을 제공할 수 있습니다.

이를 위해 본 프로젝트에서는 Unreal Engine의 Decal Component를 활용하여

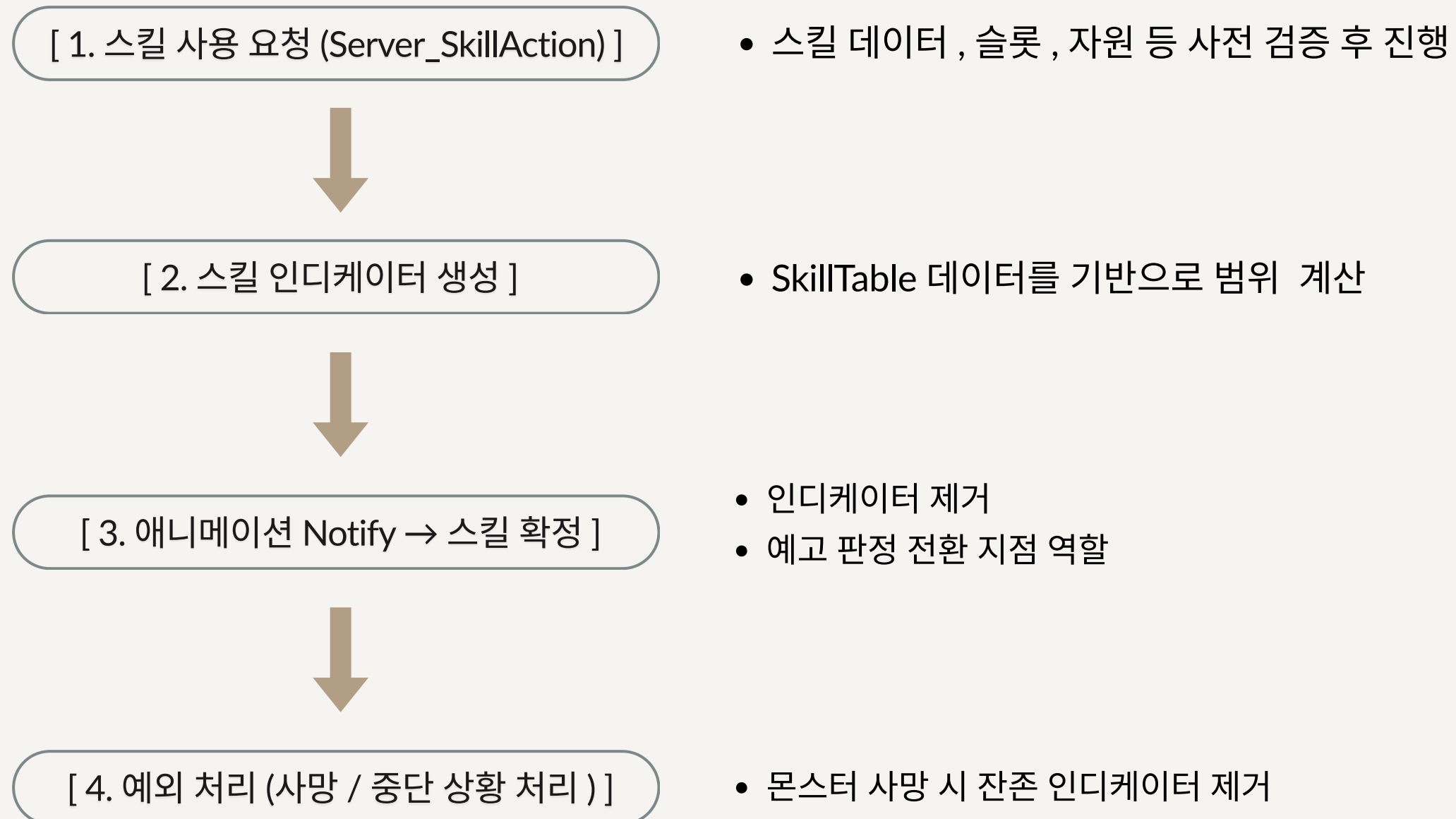
몬스터의 스킬 사거리, 범위, 형태(원형, 부채꼴, 직선형 등)를 바닥에 실시간으로 시각화 하는 스킬인디케이트 시스템을 구현하였습니다.



스킬 타입에 따라 데칼컴포넌트의 모양이 다르게 나타나는 모습



스킬인디케이트 및 데칼컴포넌트 흐름도



구조적 특징 요약

본 Skill Indicate 흐름은 다음과 같은 특징을 가집니다

- SkillComponent 중심 단일 책임
 - 판정 , 연출, 타이밍 제어를 한 흐름으로 관리
- 데이터 기반 범위 계산
 - SkillTableRow 값만으로 인디케이터 구성
- 애니메이션 기반 확정 시점
 - Notify를 통해 연출 → 판정 자연스럽게 연결
- 안정성 확보
 - 사망 / 중단 시 인디케이터 정리 보장

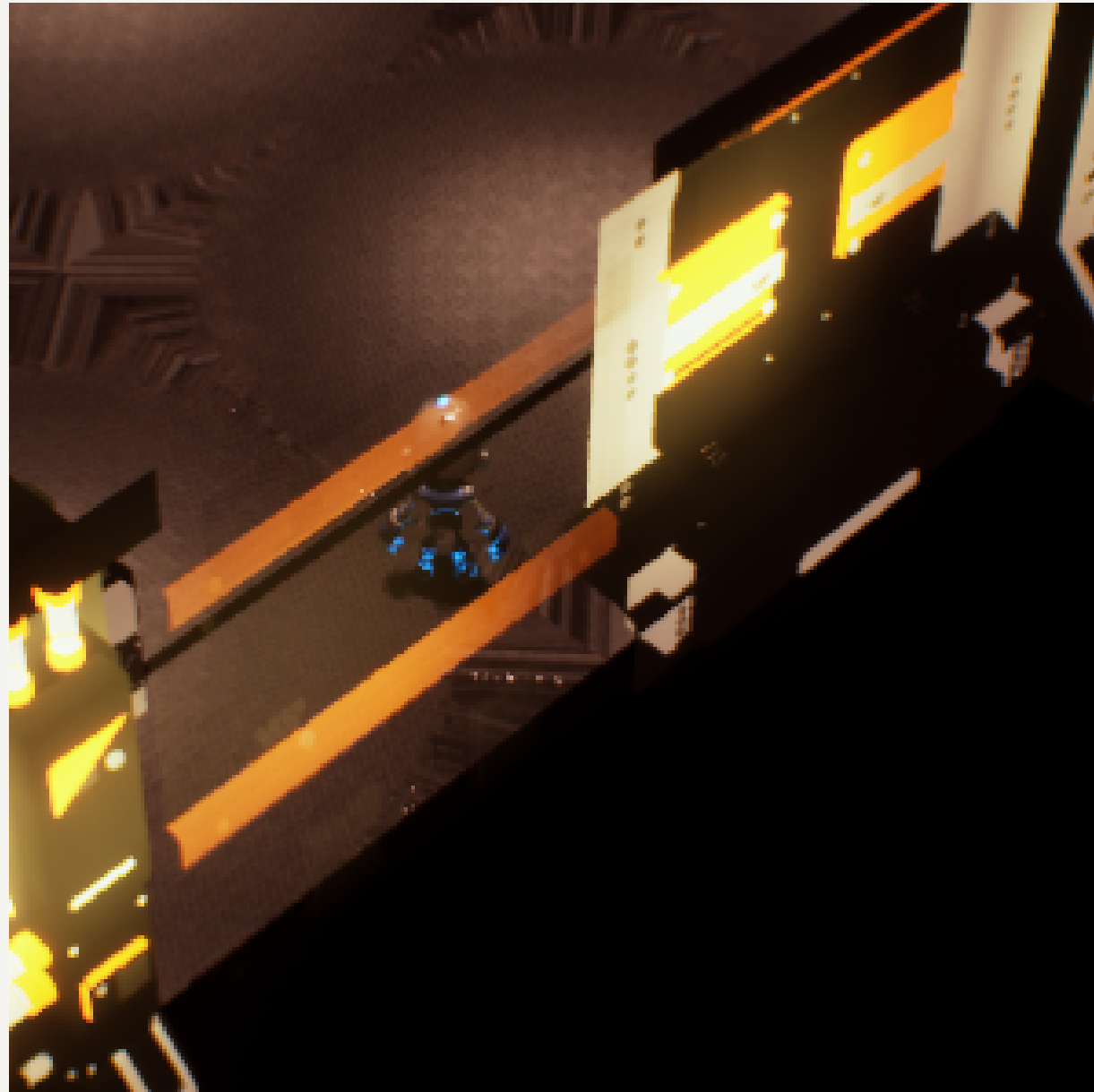
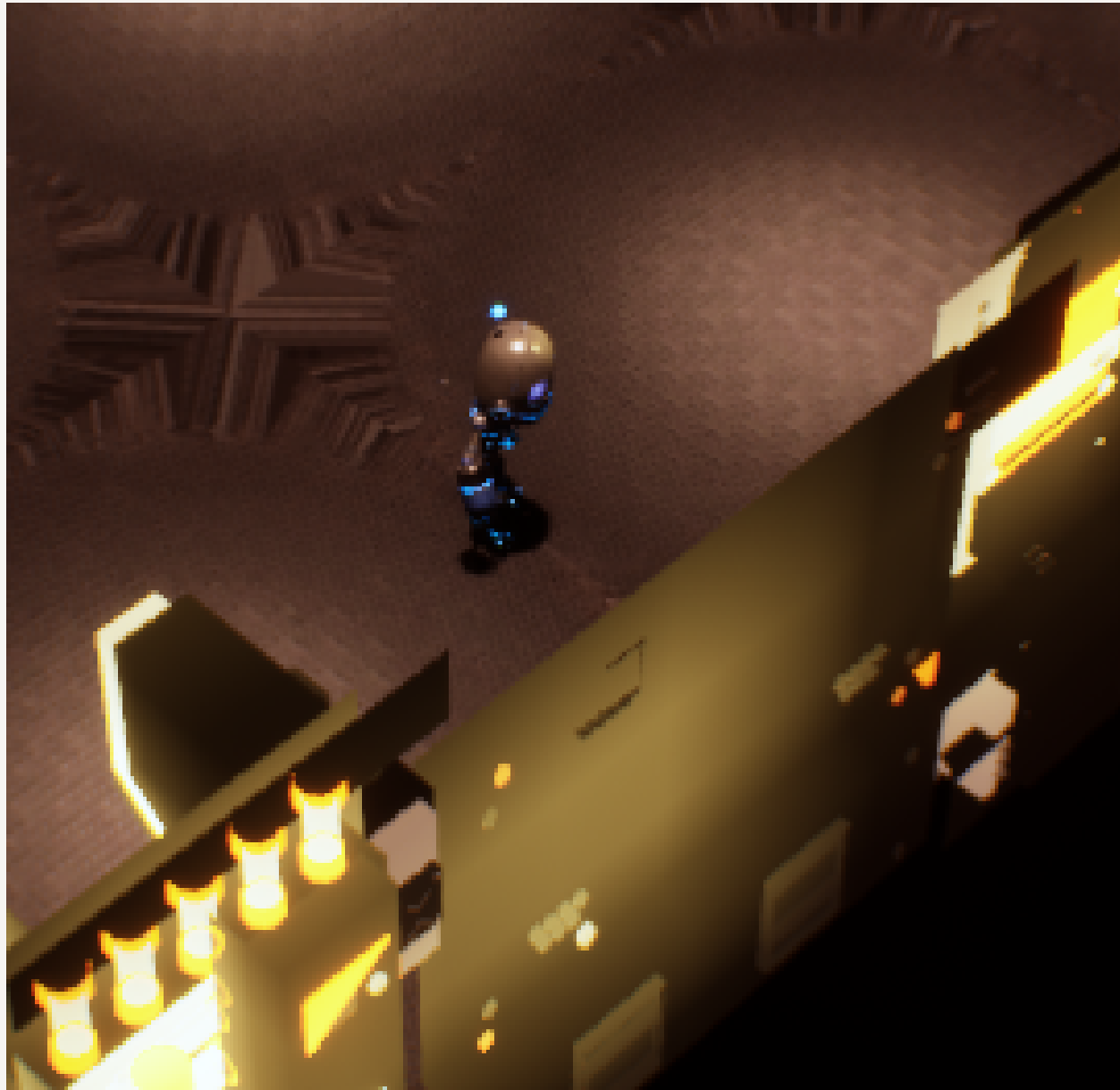


2-6 오클루전 컴포넌트



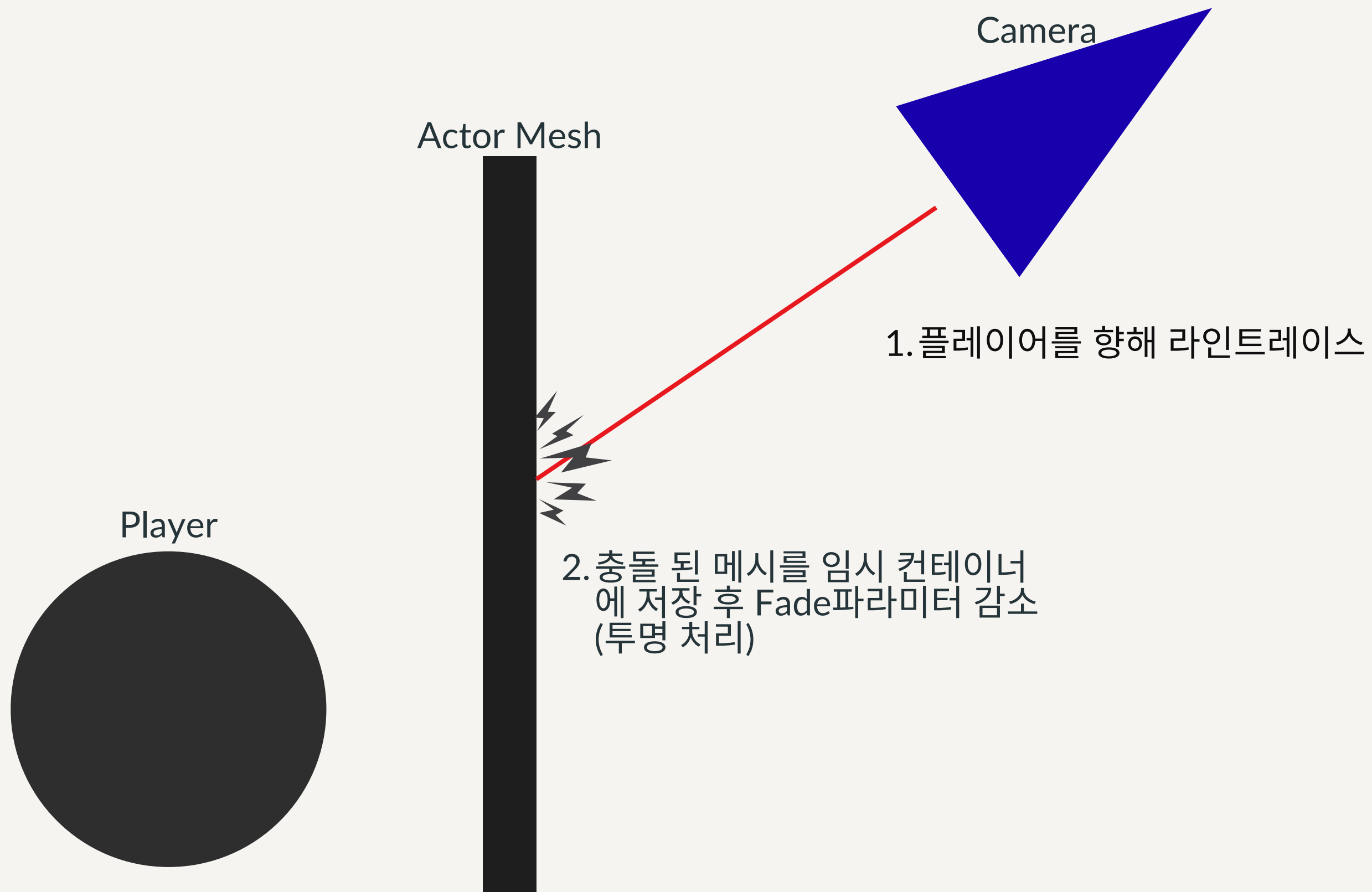
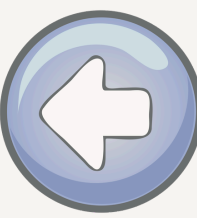
쿼터뷰 게임 특성상 카메라와 플레이어 캐릭터 사이에 액터가 위치할 경우 시야를 가려 플레이에 불편을 초래할 수 있습니다.

이를 해결하기 위해 오클루전 컴포넌트(Occlusion Component)를 구현하여 카메라에서 플레이어를 향해 라인 트레이스를 수행하고 충돌된 메시들을 일시적으로 투명 처리함으로써 항상 캐릭터가 명확히 보이도록 구성하였습니다.



플레이어가 벽 뒤로 이동

플레이어와 카메라 사이
메시가 투명해진 모습



1.시야검사

- 매 프레임 카메라에서 플레이어 방향으로 라인 트레이스를 수행합니다

2.가림 오브젝트 판별

- 라인 트레이스에 충돌된 액터를 시야 차단 요소로 분류합니다

3.투명 처리

- 충돌된 액터의 메시 머터리얼을 동적으로 투명화 합니다

4.상태 복구

- 더 이상 시야를 가리지 않는 액터는 원래 상태로 복원합니다.

- #### 5.가려진 액터 목록을 컨트롤러에 전달하여
- 마우스 입력 판정과 동기화 합니다